Introduction

Machine learning (ML) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalise to unseen data, and thus perform tasks without explicit instructions. Within a subdiscipline in machine learning, advances in the field of deep learning have allowed neural networks, a class of statistical algorithms, to surpass many previous machine learning approaches in performance.

ML finds application in many fields, including natural language processing, computer vision, speech recognition, email filtering, agriculture, and medicine. The application of ML to business problems is known as predictive analytics.

Statistics and mathematical optimisation (mathematical programming) methods comprise the foundations of machine learning. Data mining is a related field of study, focusing on exploratory data analysis (EDA) via unsupervised learning.

From a theoretical viewpoint, probably approximately correct learning provides a mathematical and statistical framework for describing machine learning. Most traditional machine learning and deep learning algorithms can be described as empirical risk minimisation under this framework.

HISTORY

The term machine learning was coined in 1959 by Arthur Samuel, an IBM employee and pioneer in the field of computer gaming and artificial intelligence. The synonym self-teaching computers was also used in this time period.

The earliest machine learning program was introduced in the 1950s when Arthur Samuel invented a computer program that calculated the winning chance in checkers for each side, but the history of machine learning roots back to decades of human desire and effort to study human cognitive processes. In 1949, Canadian psychologist Donald Hebb published the book The Organization of Behavior, in which he introduced a theoretical neural structure formed by certain interactions among nerve cells. Hebb's model of neurons interacting with one another set a groundwork for how AIs and machine learning algorithms work under nodes, or artificial neurons used by computers to communicate data. Other researchers who have studied human cognitive systems contributed to the modern machine learning technologies as well, including logician Walter Pitts and Warren McCulloch, who proposed the early mathematical models of neural networks to come up with algorithms that mirror human thought processes.

By the early 1960s, an experimental "learning machine" with punched tape memory, called Cybertron, had been developed by Raytheon Company to analyse sonar signals, electrocardiograms, and speech patterns using rudimentary reinforcement learning. It was repetitively "trained" by a human operator/teacher to recognise patterns and equipped with a "goof" button to cause it to reevaluate incorrect decisions. A representative book on research into machine learning during the 1960s was Nils Nilsson's book on Learning Machines, dealing mostly with machine learning for pattern classification. Interest related to pattern recognition continued into the 1970s, as described by Duda and Hart in 1973. In 1981, a report was given on using teaching strategies so that an artificial neural network learns to recognise 40 characters (26 letters, 10 digits, and 4 special symbols) from a computer terminal.

Tom M. Mitchell provided a widely quoted, more formal definition of the algorithms studied in the machine learning field: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E." This definition of the tasks in which machine learning is concerned offers a fundamentally operational definition rather than defining the field in cognitive terms. This follows Alan Turing's proposal in his paper "Computing Machinery and Intelligence", in which the question, "Can machines think?", is replaced with the question, "Can machines do what we (as thinking entities) can do?".

Modern-day Machine Learning algorithms are broken into 3 algorithm types: Supervised Learning Algorithms, Unsupervised Learning Algorithms, and Reinforcement Learning Algorithms.

In 2014 Ian Goodfellow and others introduced generative adversarial networks (GANs) with realistic data synthesis. By 2016 AlphaGo obtained victory against top human players using reinforcement learning techniques.

RELATIONSHIPS TO OTHER FIELDS

ARTIFICIAL INTELLIGENCE

As a scientific endeavour, machine learning grew out of the quest for artificial intelligence (AI). In the early days of AI as an academic discipline, some researchers were interested in having machines learn from data. They attempted to approach the problem with various symbolic methods, as well as what were then termed "neural networks"; these were mostly perceptrons and other models that were later found to be reinventions of the generalised linear models of statistics. Probabilistic reasoning was also employed, especially in automated medical diagnosis.

However, an increasing emphasis on the logical, knowledge-based approach caused a rift between AI and machine learning. Probabilistic systems were plagued by theoretical and practical problems of data acquisition and representation. By 1980, expert systems had come to dominate AI, and statistics was out of favour. Work on symbolic/knowledge-based learning did continue within AI, leading to inductive logic programming(ILP), but the more statistical line of research was now outside the field of AI proper, in pattern recognition and information retrieval. –710, 755 Neural networks research had been abandoned by AI and computer science around the same time. This line, too, was continued outside the AI/CS field, as "connectionism", by researchers from other disciplines, including John Hopfield, David Rumelhart, and Geoffrey Hinton. Their main success came in the mid-1980s with the reinvention of backpropagation.

Machine learning (ML), reorganised and recognised as its own field, started to flourish in the 1990s. The field changed its goal from achieving artificial intelligence to tackling solvable problems of a practical nature. It shifted focus away from the symbolic approaches it had inherited from AI, and toward methods and models borrowed from statistics, fuzzy logic, and probability theory.

DATA COMPRESSION

There is a close connection between machine learning and compression. A system that predicts the posterior probabilities of a sequence given its entire history can be used for optimal data compression (by using arithmetic coding on the output distribution). Conversely, an optimal compressor can be used for prediction (by finding the symbol that compresses best, given the previous history). This equivalence has been used as a justification for using data compression as a benchmark for "general intelligence".

An alternative view can show compression algorithms implicitly map strings into implicit feature space vectors, and compression-based similarity measures compute similarity within these feature spaces. For each compressor C(.) we define an associated vector space ℵ, such that C(.) maps an input string x, corresponding to the vector norm ||~x||. An exhaustive examination of the feature spaces underlying all compression algorithms is precluded by space; instead, feature vectors chooses to examine three representative lossless compression methods, LZW, LZ77, and PPM.

According to AIXI theory, a connection more directly explained in Hutter Prize, the best possible compression of x is the smallest possible software that generates x. For example, in that model, a zip file's compressed size includes both the zip file and the unzipping software, since you can not unzip it without both, but there may be an even smaller combined form.

Examples of AI-powered audio/video compression software include NVIDIA Maxine, AIVC. Examples of software that can perform AI-powered image compression include OpenCV, TensorFlow, MATLAB's Image Processing Toolbox (IPT) and High-Fidelity Generative Image Compression.

In unsupervised machine learning, k-means clustering can be utilized to compress data by grouping similar data points into clusters. This technique simplifies handling extensive datasets that lack

predefined labels and finds widespread use in fields such as image compression.

Data compression aims to reduce the size of data files, enhancing storage efficiency and speeding up data transmission. K-means clustering, an unsupervised machine learning algorithm, is employed to partition a dataset into a specified number of clusters, k, each represented by the centroid of its points. This process condenses extensive datasets into a more compact set of representative points. Particularly beneficial in image and signal processing, k-means clustering aids in data reduction by replacing groups of data points with their centroids, thereby preserving the core information of the original data while significantly decreasing the required storage space.

DATA MINING

Machine learning and data mining often employ the same methods and overlap significantly, but while machine learning focuses on prediction, based on known properties learned from the training data, data mining focuses on the discovery of (previously) unknown properties in the data (this is the analysis step of knowledge discovery in databases). Data mining uses many machine learning methods, but with different goals; on the other hand, machine learning also employs data mining methods as "unsupervised learning" or as a preprocessing step to improve learner accuracy. Much of the confusion between these two research communities (which do often have separate conferences and separate journals, ECML PKDD being a major exception) comes from the basic assumptions they work with: in machine learning, performance is usually evaluated with respect to the ability to reproduce known knowledge, while in knowledge discovery and data mining (KDD) the key task is the discovery of previously unknown knowledge. Evaluated with respect to known knowledge, an uninformed (unsupervised) method will easily be outperformed by other supervised methods, while in a typical KDD task, supervised methods cannot be used due to the unavailability of training data.

Machine learning also has intimate ties to optimisation: Many learning problems are formulated as minimisation of some loss function on a training set of examples. Loss functions express the discrepancy between the predictions of the model being trained and the actual problem instances (for example, in classification, one wants to assign a label to instances, and models are trained to correctly predict the preassigned labels of a set of examples).

GENERALIZATION

Characterizing the generalisation of various learning algorithms is an active topic of current research, especially for deep learning algorithms.

STATISTICS

Machine learning and statistics are closely related fields in terms of methods, but distinct in their principal goal: statistics draws population inferences from a sample, while machine learning finds generalisable predictive patterns.

Conventional statistical analyses require the a priori selection of a model most suitable for the study data set. In addition, only significant or theoretically relevant variables based on previous experience are included for analysis. In contrast, machine learning is not built on a pre-structured model; rather, the data shape the model by detecting underlying patterns. The more variables (input) used to train the model, the more accurate the ultimate model will be.

Leo Breiman distinguished two statistical modelling paradigms: data model and algorithmic model, wherein "algorithmic model" means more or less the machine learning algorithms like Random Forest.

Some statisticians have adopted methods from machine learning, leading to a combined field that they call statistical learning.

STATISTICAL PHYSICS

Analytical and computational techniques derived from deep-rooted physics of disordered systems can be extended to large-scale problems, including machine learning, e.g., to analyse the weight space of

deep neural networks. Statistical physics is thus finding applications in the area of medical diagnostics.

THEORY

A core objective of a learner is to generalise from its experience. Generalization in this context is the ability of a learning machine to perform accurately on new, unseen examples/tasks after having experienced a learning data set. The training examples come from some generally unknown probability distribution (considered representative of the space of occurrences) and the learner has to build a general model about this space that enables it to produce sufficiently accurate predictions in new cases.

The computational analysis of machine learning algorithms and their performance is a branch of theoretical computer science known as computational learning theory via the probably approximately correct learning model. Because training sets are finite and the future is uncertain, learning theory usually does not yield guarantees of the performance of algorithms. Instead, probabilistic bounds on the performance are quite common. The bias–variance decomposition is one way to quantify generalisation error.

For the best performance in the context of generalisation, the complexity of the hypothesis should match the complexity of the function underlying the data. If the hypothesis is less complex than the function, then the model has underfitted the data. If the complexity of the model is increased in response, then the training error decreases. But if the hypothesis is too complex, then the model is subject to overfitting and generalisation will be poorer.

In addition to performance bounds, learning theorists study the time complexity and feasibility of learning. In computational learning theory, a computation is considered feasible if it can be done in polynomial time. There are two kinds of time complexity results: Positive results show that a certain class of functions can be learned in polynomial time. Negative results show that certain classes cannot be learned in polynomial time.

APPROACHES

Machine learning approaches are traditionally divided into three broad categories, which correspond to learning paradigms, depending on the nature of the "signal" or "feedback" available to the learning system:

Although each algorithm has advantages and limitations, no single algorithm works for all problems.

SUPERVISED LEARNING

Supervised learning algorithms build a mathematical model of a set of data that contains both the inputs and the desired outputs. The data, known as training data, consists of a set of training examples. Each training example has one or more inputs and the desired output, also known as a supervisory signal. In the mathematical model, each training example is represented by an array or vector, sometimes called a feature vector, and the training data is represented by a matrix. Through iterative optimisation of an objective function, supervised learning algorithms learn a function that can be used to predict the output associated with new inputs. An optimal function allows the algorithm to correctly determine the output for inputs that were not a part of the training data. An algorithm that improves the accuracy of its outputs or predictions over time is said to have learned to perform that task.

Types of supervised-learning algorithms include active learning, classification and regression. Classification algorithms are used when the outputs are restricted to a limited set of values, while regression algorithms are used when the outputs can take any numerical value within a range. For example, in a classification algorithm that filters emails, the input is an incoming email, and the output is the folder in which to file the email. In contrast, regression is used for tasks such as predicting a person's height based on factors like age and genetics or forecasting future temperatures based on historical data.

Similarity learning is an area of supervised machine learning closely related to regression and classification, but the goal is to learn from examples using a similarity function that measures how similar or related two objects are. It has applications in ranking, recommendation systems, visual identity tracking, face verification, and speaker verification.

UNSUPERVISED LEARNING

Unsupervised learning algorithms find structures in data that has not been labelled, classified or categorised. Instead of responding to feedback, unsupervised learning algorithms identify commonalities in the data and react based on the presence or absence of such commonalities in each new piece of data. Central applications of unsupervised machine learning include clustering, dimensionality reduction, and density estimation.

Cluster analysis is the assignment of a set of observations into subsets (called clusters) so that observations within the same cluster are similar according to one or more predesignated criteria, while observations drawn from different clusters are dissimilar. Different clustering techniques make different assumptions on the structure of the data, often defined by some similarity metric and evaluated, for example, by internal compactness, or the similarity between members of the same cluster, and separation, the difference between clusters. Other methods are based on estimated density and graph connectivity.

A special type of unsupervised learning called, self-supervised learning involves training a model by generating the supervisory signal from the data itself.

SEMI-SUPERVISED LEARNING

Semi-supervised learning falls between unsupervised learning (without any labelled training data) and supervised learning (with completely labelled training data). Some of the training examples are missing training labels, yet many machine-learning researchers have found that unlabelled data, when used in conjunction with a small amount of labelled data, can produce a considerable improvement in learning accuracy.

In weakly supervised learning, the training labels are noisy, limited, or imprecise; however, these labels are often cheaper to obtain, resulting in larger effective training sets.

REINFORCEMENT LEARNING

Reinforcement learning is an area of machine learning concerned with how software agents ought to take actions in an environment to maximise some notion of cumulative reward. Due to its generality, the field is studied in many other disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimisation, multi-agent systems, swarm intelligence, statistics and genetic algorithms. In reinforcement learning, the environment is typically represented as a Markov decision process (MDP). Many reinforcement learning algorithms use dynamic programming techniques. Reinforcement learning algorithms do not assume knowledge of an exact mathematical model of the MDP and are used when exact models are infeasible. Reinforcement learning algorithms are used in autonomous vehicles or in learning to play a game against a human opponent.

DIMENSIONALITY REDUCTION

Dimensionality reduction is a process of reducing the number of random variables under consideration by obtaining a set of principal variables. In other words, it is a process of reducing the dimension of the feature set, also called the "number of features". Most of the dimensionality reduction techniques can be considered as either feature elimination or extraction. One of the popular methods of dimensionality reduction is principal component analysis (PCA). PCA involves changing higher-dimensional data (e.g., 3D) to a smaller space (e.g., 2D). The manifold hypothesis proposes that high-dimensional data sets lie along low-dimensional manifolds, and many dimensionality reduction techniques make this assumption, leading to the areas of manifold learning and manifold regularisation.

OTHER TYPES

Other approaches have been developed which do not fit neatly into this three-fold categorisation, and sometimes more than one is used by the same machine learning system. For example, topic modelling, meta-learning.

Self-learning, as a machine learning paradigm, was introduced in 1982 along with a neural network capable of self-learning, named crossbar adaptive array (CAA). It gives a solution to the problem learning without any external reward, by introducing emotion as an internal reward. Emotion is used as a state evaluation of a self-learning agent. The CAA self-learning algorithm computes, in a crossbar fashion, both decisions about actions and emotions (feelings) about consequence situations. The system is driven by the interaction between cognition and emotion. The self-learning algorithm updates a memory matrix W =||w(a,s)|| such that in each iteration executes the following machine learning routine:

It is a system with only one input, situation, and only one output, action (or behaviour) a. There is neither a separate reinforcement input nor an advice input from the environment. The backpropagated value (secondary reinforcement) is the emotion toward the consequence situation. The CAA exists in two environments, one is the behavioural environment where it behaves, and the other is the genetic environment, wherefrom it initially and only once receives initial emotions about situations to be encountered in the behavioural environment. After receiving the genome (species) vector from the genetic environment, the CAA learns a goal-seeking behaviour in an environment that contains both desirable and undesirable situations.

Several learning algorithms aim at discovering better representations of the inputs provided during training. Classic examples include principal component analysis and cluster analysis. Feature learning algorithms, also called representation learning algorithms, often attempt to preserve the information in their input but also transform it in a way that makes it useful, often as a pre-processing step before performing classification or predictions. This technique allows reconstruction of the inputs coming from the unknown data-generating distribution, while not being necessarily faithful to configurations that are implausible under that distribution. This replaces manual feature engineering, and allows a machine to both learn the features and use them to perform a specific task.

Feature learning can be either supervised or unsupervised. In supervised feature learning, features are learned using labelled input data. Examples include artificial neural networks, multilayer perceptrons, and supervised dictionary learning. In unsupervised feature learning, features are learned with unlabelled input data. Examples include dictionary learning, independent component analysis, autoencoders, matrix factorisation and various forms of clustering.

Manifold learning algorithms attempt to do so under the constraint that the learned representation is low-dimensional. Sparse coding algorithms attempt to do so under the constraint that the learned representation is sparse, meaning that the mathematical model has many zeros. Multilinear subspace learning algorithms aim to learn low-dimensional representations directly from tensor representations for multidimensional data, without reshaping them into higher-dimensional vectors. Deep learning algorithms discover multiple levels of representation, or a hierarchy of features, with higher-level, more abstract features defined in terms of (or generating) lower-level features. It has been argued that an intelligent machine learns a representation that disentangles the underlying factors of variation that explain the observed data.

Feature learning is motivated by the fact that machine learning tasks such as classification often require input that is mathematically and computationally convenient to process. However, real-world data such as images, video, and sensory data have not yielded attempts to algorithmically define specific features. An alternative is to discover such features or representations through examination, without relying on explicit algorithms.

Sparse dictionary learning is a feature learning method where a training example is represented as a linear combination of basis functions and assumed to be a sparse matrix. The method is strongly NP-hard and difficult to solve approximately. A popular heuristic method for sparse dictionary learning is the k-SVD algorithm. Sparse dictionary learning has been applied in several contexts. In classification, the problem is to determine the class to which a previously unseen training example belongs. For a dictionary where each class has already been built, a new training example is associated with the class that is best sparsely represented by the corresponding dictionary. Sparse

dictionary learning has also been applied in image denoising. The key idea is that a clean image patch can be sparsely represented by an image dictionary, but the noise cannot.

In data mining, anomaly detection, also known as outlier detection, is the identification of rare items, events or observations that raise suspicions by differing significantly from the majority of the data. Typically, the anomalous items represent an issue such as bank fraud, a structural defect, medical problems or errors in a text. Anomalies are referred to as outliers, novelties, noise, deviations and exceptions.

In particular, in the context of abuse and network intrusion detection, the interesting objects are often not rare, but unexpected bursts of inactivity. This pattern does not adhere to the common statistical definition of an outlier as a rare object. Many outlier detection methods (in particular, unsupervised algorithms) will fail on such data unless aggregated appropriately. Instead, a cluster analysis algorithm may be able to detect the micro-clusters formed by these patterns.

Three broad categories of anomaly detection techniques exist. Unsupervised anomaly detection techniques detect anomalies in an unlabelled test data set under the assumption that the majority of the instances in the data set are normal, by looking for instances that seem to fit the least to the remainder of the data set. Supervised anomaly detection techniques require a data set that has been labelled as "normal" and "abnormal" and involves training a classifier (the key difference from many other statistical classification problems is the inherently unbalanced nature of outlier detection). Semi-supervised anomaly detection techniques construct a model representing normal behaviour from a given normal training data set and then test the likelihood of a test instance being generated by the model.

Robot learning is inspired by a multitude of machine learning methods, starting from supervised learning, reinforcement learning, and finally meta-learning (e.g. MAML).

Association rule learning is a rule-based machine learning method for discovering relationships between variables in large databases. It is intended to identify strong rules discovered in databases using some measure of "interestingness".

Rule-based machine learning is a general term for any machine learning method that identifies, learns, or evolves "rules" to store, manipulate or apply knowledge. The defining characteristic of a rule-based machine learning algorithm is the identification and utilisation of a set of relational rules that collectively represent the knowledge captured by the system. This is in contrast to other machine learning algorithms that commonly identify a singular model that can be universally applied to any instance in order to make a prediction. Rule-based machine learning approaches include learning classifier systems, association rule learning, and artificial immune systems.

Based on the concept of strong rules, Rakesh Agrawal, Tomasz Imieli■ski and Arun Swami introduced association rules for discovering regularities between products in large-scale transaction data recorded by point-of-sale (POS) systems in supermarkets. For example, the rule $\{ o n i o n s , p o t a t o e s \} \Rightarrow \{ b u r g e r \}$ {\displaystyle \{\mathrm {onions,potatoes} \}\Rightarrow \{\mathrm {burger} \}} found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, they are likely to also buy hamburger meat. Such information can be used as the basis for decisions about marketing activities such as promotional pricing or product placements. In addition to market basket analysis, association rules are employed today in application areas including Web usage mining, intrusion detection, continuous production, and bioinformatics. In contrast with sequence mining, association rule learning typically does not consider the order of items either within a transaction or across transactions.

Learning classifier systems (LCS) are a family of rule-based machine learning algorithms that combine a discovery component, typically a genetic algorithm, with a learning component, performing either supervised learning, reinforcement learning, or unsupervised learning. They seek to identify a set of context-dependent rules that collectively store and apply knowledge in a piecewise manner to make predictions.

Inductive logic programming (ILP) is an approach to rule learning using logic programming as a uniform representation for input examples, background knowledge, and hypotheses. Given an encoding of the known background knowledge and a set of examples represented as a logical database of facts, an ILP system will derive a hypothesized logic program that entails all positive and no negative examples. Inductive programming is a related field that considers any kind of programming language for representing hypotheses (and not only logic programming), such as functional programs.

Inductive logic programming is particularly useful in bioinformatics and natural language processing. Gordon Plotkin and Ehud Shapiro laid the initial theoretical foundation for inductive machine learning in a logical setting. Shapiro built their first implementation (Model Inference System) in 1981: a Prolog program that inductively inferred logic programs from positive and negative examples. The term inductive here refers to philosophical induction, suggesting a theory to explain observed facts, rather than mathematical induction, proving a property for all members of a well-ordered set.

MODELS

A machine learning model is a type of mathematical model that, once "trained" on a given dataset, can be used to make predictions or classifications on new data. During training, a learning algorithm iteratively adjusts the model's internal parameters to minimise errors in its predictions. By extension, the term "model" can refer to several levels of specificity, from a general class of models and their associated learning algorithms to a fully trained model with all its internal parameters tuned.

Various types of models have been used and researched for machine learning systems, picking the best model for a task is called model selection.

ARTIFICIAL NEURAL NETWORKS

Artificial neural networks (ANNs), or connectionist systems, are computing systems vaguely inspired by the biological neural networks that constitute animal brains. Such systems "learn" to perform tasks by considering examples, generally without being programmed with any task-specific rules.

An ANN is a model based on a collection of connected units or nodes called "artificial neurons", which loosely model the neurons in a biological brain. Each connection, like the synapses in a biological brain, can transmit information, a "signal", from one artificial neuron to another. An artificial neuron that receives a signal can process it and then signal additional artificial neurons connected to it. In common ANN implementations, the signal at a connection between artificial neurons is a real number, and the output of each artificial neuron is computed by some non-linear function of the sum of its inputs. The connections between artificial neurons are called "edges". Artificial neurons and edges typically have a weight that adjusts as learning proceeds. The weight increases or decreases the strength of the signal at a connection. Artificial neurons may have a threshold such that the signal is only sent if the aggregate signal crosses that threshold. Typically, artificial neurons are aggregated into layers. Different layers may perform different kinds of transformations on their inputs. Signals travel from the first layer (the input layer) to the last layer (the output layer), possibly after traversing the layers multiple times.

The original goal of the ANN approach was to solve problems in the same way that a human brain would. However, over time, attention moved to performing specific tasks, leading to deviations from biology. Artificial neural networks have been used on a variety of tasks, including computer vision, speech recognition, machine translation, social network filtering, playing board and video games and medical diagnosis.

Deep learning consists of multiple hidden layers in an artificial neural network. This approach tries to model the way the human brain processes light and sound into vision and hearing. Some successful applications of deep learning are computer vision and speech recognition.

DECISION TREES

Decision tree learning uses a decision tree as a predictive model to go from observations about an item (represented in the branches) to conclusions about the item's target value (represented in the leaves). It is one of the predictive modelling approaches used in statistics, data mining, and machine learning. Tree models where the target variable can take a discrete set of values are called classification trees; in these tree structures, leaves represent class labels, and branches represent conjunctions of features that lead to those class labels. Decision trees where the target variable can take continuous values (typically real numbers) are called regression trees. In decision analysis, a decision tree can be used to visually and explicitly represent decisions and decision making. In data mining, a decision tree

describes data, but the resulting classification tree can be an input for decision-making.

RANDOM FOREST REGRESSION

Random forest regression (RFR) falls under the umbrella of decision tree-based models. RFR is an ensemble learning method that builds multiple decision trees and averages their predictions to improve accuracy and to avoid overfitting. To build decision trees, RFR uses bootstrapped sampling; for instance, each decision tree is trained on random data from the training set. This random selection of RFR for training enables the model to reduce biased predictions and achieve a higher degree of accuracy. RFR generates independent decision trees, and it can work on single-output data as well as multiple regressor tasks. This makes RFR compatible to be use in various applications.

SUPPORT-VECTOR MACHINES

Support-vector machines (SVMs), also known as support-vector networks, are a set of related supervised learning methods used for classification and regression. Given a set of training examples, each marked as belonging to one of two categories, an SVM training algorithm builds a model that predicts whether a new example falls into one category. An SVM training algorithm is a non-probabilistic, binary, linear classifier, although methods such as Platt scaling exist to use SVM in a probabilistic classification setting. In addition to performing linear classification, SVMs can efficiently perform a non-linear classification using what is called the kernel trick, implicitly mapping their inputs into high-dimensional feature spaces.

REGRESSION ANALYSIS

Regression analysis encompasses a large variety of statistical methods to estimate the relationship between input variables and their associated features. Its most common form is linear regression, where a single line is drawn to best fit the given data according to a mathematical criterion such as ordinary least squares. The latter is often extended by regularisation methods to mitigate overfitting and bias, as in ridge regression. When dealing with non-linear problems, go-to models include polynomial regression (for example, used for trendline fitting in Microsoft Excel), logistic regression (often used in statistical classification) or even kernel regression, which introduces non-linearity by taking advantage of the kernel trick to implicitly map input variables to higher-dimensional space.

Multivariate linear regression extends the concept of linear regression to handle multiple dependent variables simultaneously. This approach estimates the relationships between a set of input variables and several output variables by fitting a multidimensional linear model. It is particularly useful in scenarios where outputs are interdependent or share underlying patterns, such as predicting multiple economic indicators or reconstructing images, which are inherently multi-dimensional.

BAYESIAN NETWORKS

A Bayesian network, belief network, or directed acyclic graphical model is a probabilistic graphical model that represents a set of random variables and their conditional independence with a directed acyclic graph (DAG). For example, a Bayesian network could represent the probabilistic relationships between diseases and symptoms. Given symptoms, the network can be used to compute the probabilities of the presence of various diseases. Efficient algorithms exist that perform inference and learning. Bayesian networks that model sequences of variables, like speech signals or protein sequences, are called dynamic Bayesian networks. Generalisations of Bayesian networks that can represent and solve decision problems under uncertainty are called influence diagrams.

GAUSSIAN PROCESSES

A Gaussian process is a stochastic process in which every finite collection of the random variables in the process has a multivariate normal distribution, and it relies on a pre-defined covariance function, or kernel, that models how pairs of points relate to each other depending on their locations.

Given a set of observed points, or input–output examples, the distribution of the (unobserved) output of a new point as a function of its input data can be directly computed by looking at the observed points and the covariances between those points and the new, unobserved point.

Gaussian processes are popular surrogate models in Bayesian optimisation used to do hyperparameter optimisation.

GENETIC ALGORITHMS

A genetic algorithm (GA) is a search algorithm and heuristic technique that mimics the process of natural selection, using methods such as mutation and crossover to generate new genotypes in the hope of finding good solutions to a given problem. In machine learning, genetic algorithms were used in the 1980s and 1990s. Conversely, machine learning techniques have been used to improve the performance of genetic and evolutionary algorithms.

BELIEF FUNCTIONS

The theory of belief functions, also referred to as evidence theory or Dempster–Shafer theory, is a general framework for reasoning with uncertainty, with understood connections to other frameworks such as probability, possibility and imprecise probability theories. These theoretical frameworks can be thought of as a kind of learner and have some analogous properties of how evidence is combined (e.g., Dempster's rule of combination), just like how in a pmf-based Bayesian approach would combine probabilities. However, there are many caveats to these beliefs functions when compared to Bayesian approaches to incorporate ignorance and uncertainty quantification. These belief function approaches that are implemented within the machine learning domain typically leverage a fusion approach of various ensemble methods to better handle the learner's decision boundary, low samples, and ambiguous class issues that standard machine learning approach tend to have difficulty resolving. However, the computational complexity of these algorithms is dependent on the number of propositions (classes), and can lead to a much higher computation time when compared to other machine learning approaches.

RULE-BASED MODELS

Rule-based machine learning (RBML) is a branch of machine learning that automatically discovers and learns 'rules' from data. It provides interpretable models, making it useful for decision-making in fields like healthcare, fraud detection, and cybersecurity. Key RBML techniques includes learning classifier systems, association rule learning, artificial immune systems, and other similar models. These methods extract patterns from data and evolve rules over time.

TRAINING MODELS

Typically, machine learning models require a high quantity of reliable data to perform accurate predictions. When training a machine learning model, machine learning engineers need to target and collect a large and representative sample of data. Data from the training set can be as varied as a corpus of text, a collection of images, sensor data, and data collected from individual users of a service. Overfitting is something to watch out for when training a machine learning model. Trained models derived from biased or non-evaluated data can result in skewed or undesired predictions. Biased models may result in detrimental outcomes, thereby furthering the negative impacts on society or objectives. Algorithmic bias is a potential result of data not being fully prepared for training. Machine learning ethics is becoming a field of study and, notably, becoming integrated within machine learning engineering teams.

Federated learning is an adapted form of distributed artificial intelligence to train machine learning models that decentralises the training process, allowing for users' privacy to be maintained by not needing to send their data to a centralised server. This also increases efficiency by decentralising the training process to many devices. For example, Gboard uses federated machine learning to train search query prediction models on users' mobile phones without having to send individual searches back to Google.

APPLICATIONS

There are many applications for machine learning, including:

In 2006, the media-services provider Netflix held the first "Netflix Prize" competition to find a program to better predict user preferences and improve the accuracy of its existing Cinematch movie recommendation algorithm by at least 10%. A joint team made up of researchers from AT&T; Labs-Research in collaboration with the teams Big Chaos and Pragmatic Theory built an ensemble model to win the Grand Prize in 2009 for $1 million. Shortly after the prize was awarded, Netflix realised that viewers' ratings were not the best indicators of their viewing patterns ("everything is a recommendation") and they changed their recommendation engine accordingly. In 2010, an article in The Wall Street Journal noted the use of machine learning by Rebellion Research to predict the 2008 financial crisis. In 2012, co-founder of Sun Microsystems, Vinod Khosla, predicted that 80% of medical doctors jobs would be lost in the next two decades to automated machine learning medical diagnostic software. In 2014, it was reported that a machine learning algorithm had been applied in the field of art history to study fine art paintings and that it may have revealed previously unrecognised influences among artists. In 2019 Springer Nature published the first research book created using machine learning. In 2020, machine learning technology was used to help make diagnoses and aid researchers in developing a cure for COVID-19. Machine learning was recently applied to predict the pro-environmental behaviour of travellers. Recently, machine learning technology was also applied to optimise smartphone's performance and thermal behaviour based on the user's interaction with the phone. When applied correctly, machine learning algorithms (MLAs) can utilise a wide range of company characteristics to predict stock returns without overfitting. By employing effective feature engineering and combining forecasts, MLAs can generate results that far surpass those obtained from basic linear techniques like OLS.

Recent advancements in machine learning have extended into the field of quantum chemistry, where novel algorithms now enable the prediction of solvent effects on chemical reactions, thereby offering new tools for chemists to tailor experimental conditions for optimal outcomes.

Machine Learning is becoming a useful tool to investigate and predict evacuation decision-making in large-scale and small-scale disasters. Different solutions have been tested to predict if and when householders decide to evacuate during wildfires and hurricanes. Other applications have been focusing on pre evacuation decisions in building fires.

LIMITATIONS

Although machine learning has been transformative in some fields, machine-learning programs often fail to deliver expected results. Reasons for this are numerous: lack of (suitable) data, lack of access to the data, data bias, privacy problems, badly chosen tasks and algorithms, wrong tools and people, lack of resources, and evaluation problems.

The "black box theory" poses another yet significant challenge. Black box refers to a situation where the algorithm or the process of producing an output is entirely opaque, meaning that even the coders of the algorithm cannot audit the pattern that the machine extracted from the data. The House of Lords Select Committee, which claimed that such an "intelligence system" that could have a "substantial impact on an individual's life" would not be considered acceptable unless it provided "a full and satisfactory explanation for the decisions" it makes.

In 2018, a self-driving car from Uber failed to detect a pedestrian, who was killed after a collision. Attempts to use machine learning in healthcare with the IBM Watson system failed to deliver even after years of time and billions of dollars invested. Microsoft's Bing Chat chatbot has been reported to produce hostile and offensive response against its users.

Machine learning has been used as a strategy to update the evidence related to a systematic review and increased reviewer burden related to the growth of biomedical literature. While it has improved with training sets, it has not yet developed sufficiently to reduce the workload burden without limiting the

necessary sensitivity for the findings research itself.

EXPLAINABILITY

Explainable AI (XAI), or Interpretable AI, or Explainable Machine Learning (XML), is artificial intelligence (AI) in which humans can understand the decisions or predictions made by the AI. It contrasts with the "black box" concept in machine learning where even its designers cannot explain why an AI arrived at a specific decision. By refining the mental models of users of AI-powered systems and dismantling their misconceptions, XAI promises to help users perform more effectively. XAI may be an implementation of the social right to explanation.

OVERFITTING

Settling on a bad, overly complex theory gerrymandered to fit all the past training data is known as overfitting. Many systems attempt to reduce overfitting by rewarding a theory in accordance with how well it fits the data but penalising the theory in accordance with how complex the theory is.

OTHER LIMITATIONS AND VULNERABILITIES

Learners can also be disappointed by "learning the wrong lesson". A toy example is that an image classifier trained only on pictures of brown horses and black cats might conclude that all brown patches are likely to be horses. A real-world example is that, unlike humans, current image classifiers often do not primarily make judgments from the spatial relationship between components of the picture, and they learn relationships between pixels that humans are oblivious to, but that still correlate with images of certain types of real objects. Modifying these patterns on a legitimate image can result in "adversarial" images that the system misclassifies.

Adversarial vulnerabilities can also result in nonlinear systems or from non-pattern perturbations. For some systems, it is possible to change the output by only changing a single adversarially chosen pixel. Machine learning models are often vulnerable to manipulation or evasion via adversarial machine learning.

Researchers have demonstrated how backdoors can be placed undetectably into classifying (e.g., for categories "spam" and "not spam" of posts) machine learning models that are often developed or trained by third parties. Parties can change the classification of any input, including in cases for which a type of data/software transparency is provided, possibly including white-box access.

MODEL ASSESSMENTS

Classification of machine learning models can be validated by accuracy estimation techniques like the holdout method, which splits the data into a training and test set (conventionally 2/3 training set and 1/3 test set designation) and evaluates the performance of the training model on the test set. In comparison, the K-fold-cross-validation method randomly partitions the data into K subsets and then K experiments are performed each considering 1 subset for evaluation and the remaining K-1 subsets for training the model. In addition to the holdout and cross-validation methods, bootstrap, which samples n instances with replacement from the dataset, can be used to assess model accuracy.

In addition to overall accuracy, investigators frequently report sensitivity and specificity, meaning true positive rate (TPR) and true negative rate (TNR), respectively. Similarly, investigators sometimes report the false positive rate (FPR) as well as the false negative rate (FNR). However, these rates are ratios that fail to reveal their numerators and denominators. Receiver operating characteristic (ROC), along with the accompanying Area Under the ROC Curve (AUC), offer additional tools for classification model assessment. Higher AUC is associated with a better performing model.

ETHICS

The ethics of artificial intelligence covers a broad range of topics within AI that are considered to have particular ethical stakes. This includes algorithmic biases, fairness, automated decision-making, accountability, privacy, and regulation. It also covers various emerging or potential future challenges such as machine ethics (how to make machines that behave ethically), lethal autonomous weapon systems, arms race dynamics, AI safety and alignment, technological unemployment, AI-enabled misinformation, how to treat certain AI systems if they have a moral status (AI welfare and rights), artificial superintelligence and existential risks.

BIAS

Different machine learning approaches can suffer from different data biases. A machine learning system trained specifically on current customers may not be able to predict the needs of new customer groups that are not represented in the training data. When trained on human-made data, machine learning is likely to pick up the constitutional and unconscious biases already present in society.

Systems that are trained on datasets collected with biases may exhibit these biases upon use (algorithmic bias), thus digitising cultural prejudices. For example, in 1988, the UK's Commission for Racial Equality found that St. George's Medical School had been using a computer program trained from data of previous admissions staff and this program had denied nearly 60 candidates who were found to either be women or have non-European-sounding names. Using job hiring data from a firm with racist hiring policies may lead to a machine learning system duplicating the bias by scoring job applicants by similarity to previous successful applicants. Another example includes predictive policing company Geolitica's predictive algorithm that resulted in "disproportionately high levels of over-policing in low-income and minority communities" after being trained with historical crime data.

While responsible collection of data and documentation of algorithmic rules used by a system is considered a critical part of machine learning, some researchers blame the lack of participation and representation of minority populations in the field of AI for machine learning's vulnerability to biases. In fact, according to research carried out by the Computing Research Association in 2021, "female faculty make up just 16.1%" of all faculty members who focus on AI among several universities around the world. Furthermore, among the group of "new U.S. resident AI PhD graduates," 45% identified as white, 22.4% as Asian, 3.2% as Hispanic, and 2.4% as African American, which further demonstrates a lack of diversity in the field of AI.

Language models learned from data have been shown to contain human-like biases. Because human languages contain biases, machines trained on language corpora will necessarily also learn these biases. In 2016, Microsoft tested Tay, a chatbot that learned from Twitter, and it quickly picked up racist and sexist language.

In an experiment carried out by ProPublica, an investigative journalism organisation, a machine learning algorithm's insight into the recidivism rates among prisoners falsely flagged "black defendants high risk twice as often as white defendants". In 2015, Google Photos once tagged a couple of black people as gorillas, which caused controversy. The gorilla label was subsequently removed, and in 2023, it still cannot recognise gorillas. Similar issues with recognising non-white people have been found in many other systems.

Because of such challenges, the effective use of machine learning may take longer to be adopted in other domains. Concern for fairness in machine learning, that is, reducing bias in machine learning and propelling its use for human good, is increasingly expressed by artificial intelligence scientists, including Fei-Fei Li, who said that " here's nothing artificial about AI. It's inspired by people, it's created by people, and—most importantly—it impacts people. It is a powerful tool we are only just beginning to understand, and that is a profound responsibility."

FINANCIAL INCENTIVES

There are concerns among health care professionals that these systems might not be designed in the public's interest but as income-generating machines. This is especially true in the United States, where there is a long-standing ethical dilemma of improving health care, but also increasing profits. For example, the algorithms could be designed to provide patients with unnecessary tests or medication in which the algorithm's proprietary owners hold stakes. There is potential for machine learning in health

care to provide professionals with an additional tool to diagnose, medicate, and plan recovery paths for patients, but this requires these biases to be mitigated.

HARDWARE

Since the 2010s, advances in both machine learning algorithms and computer hardware have led to more efficient methods for training deep neural networks (a particular narrow subdomain of machine learning) that contain many layers of nonlinear hidden units. By 2019, graphics processing units (GPUs), often with AI-specific enhancements, had displaced CPUs as the dominant method of training large-scale commercial cloud AI. OpenAI estimated the hardware compute used in the largest deep learning projects from AlexNet (2012) to AlphaZero (2017), and found a 300,000-fold increase in the amount of compute required, with a doubling-time trendline of 3.4 months.

TENSOR PROCESSING UNITS (TPUS)

Tensor Processing Units (TPUs) are specialised hardware accelerators developed by Google specifically for machine learning workloads. Unlike general-purpose GPUs and FPGAs, TPUs are optimised for tensor computations, making them particularly efficient for deep learning tasks such as training and inference. They are widely used in Google Cloud AI services and large-scale machine learning models like Google's DeepMind AlphaFold and large language models. TPUs leverage matrix multiplication units and high-bandwidth memory to accelerate computations while maintaining energy efficiency. Since their introduction in 2016, TPUs have become a key component of AI infrastructure, especially in cloud-based environments.

NEUROMORPHIC COMPUTING

Neuromorphic computing refers to a class of computing systems designed to emulate the structure and functionality of biological neural networks. These systems may be implemented through software-based simulations on conventional hardware or through specialised hardware architectures.

A physical neural network is a specific type of neuromorphic hardware that relies on electrically adjustable materials, such as memristors, to emulate the function of neural synapses. The term "physical neural network" highlights the use of physical hardware for computation, as opposed to software-based implementations. It broadly refers to artificial neural networks that use materials with adjustable resistance to replicate neural synapses.

EMBEDDED MACHINE LEARNING

Embedded machine learning is a sub-field of machine learning where models are deployed on embedded systems with limited computing resources, such as wearable computers, edge devices and microcontrollers. Running models directly on these devices eliminates the need to transfer and store data on cloud servers for further processing, thereby reducing the risk of data breaches, privacy leaks and theft of intellectual property, personal data and business secrets. Embedded machine learning can be achieved through various techniques, such as hardware acceleration, approximate computing, and model optimisation. Common optimisation techniques include pruning, quantisation, knowledge distillation, low-rank factorisation, network architecture search, and parameter sharing.

SOFTWARE

Software suites containing a variety of machine learning algorithms include the following:

FREE AND OPEN-SOURCE SOFTWARE

PROPRIETARY SOFTWARE WITH FREE AND OPEN-SOURCE EDITIONS

PROPRIETARY SOFTWARE

JOURNALS

CONFERENCES

Machine Learning is a peer-reviewed scientific journal, published since 1986.

In 2001, forty editors and members of the editorial board of Machine Learning resigned in order to support the Journal of Machine Learning Research (JMLR), saying that in the era of the internet, it was detrimental for researchers to continue publishing their papers in expensive journals with pay-access archives. Instead, they wrote, they supported the model of JMLR, in which authors retained copyright over their papers and archives were freely available on the internet.

Following the mass resignation, Kluwer changed their publishing policy to allow authors to self-archive their papers online after peer-review.

ABSTRACTING AND INDEXING

The journal is abstracted and indexed in several databases, for example in:

SELECTED ARTICLES

Statistical learning is the ability for humans and other animals to extract statistical regularities from the world around them to learn about the environment. Although statistical learning is now thought to be a generalized learning mechanism, the phenomenon was first identified in human infant language acquisition.

The earliest evidence for these statistical learning abilities comes from a study by Jenny Saffran, Richard Aslin, and Elissa Newport, in which 8-month-old infants were presented with nonsense streams of monotone speech. Each stream was composed of four three-syllable "pseudowords" that were repeated randomly. After exposure to the speech streams for two minutes, infants reacted differently to hearing "pseudowords" as opposed to "nonwords" from the speech stream, where nonwords were composed of the same syllables that the infants had been exposed to, but in a different order. This suggests that infants are able to learn statistical relationships between syllables even with very limited exposure to a language. That is, infants learn which syllables are always paired together and which ones only occur together relatively rarely, suggesting that they are parts of two different units. This method of learning is thought to be one way that children learn which groups of syllables form individual words.

Since the initial discovery of the role of statistical learning in lexical acquisition, the same mechanism has been proposed for elements of phonological acquisition, and syntactical acquisition, as well as in non-linguistic domains. Further research has also indicated that statistical learning is likely a domain-general and even species-general learning mechanism, occurring for visual as well as auditory information, and in both primates and non-primates.

LEXICAL ACQUISITION

The role of statistical learning in language acquisition has been particularly well documented in the area of lexical acquisition. One important contribution to infants' understanding of segmenting words from a continuous stream of speech is their ability to recognize statistical regularities of the speech heard in their environments. Although many factors play an important role, this specific mechanism is powerful and can operate over a short time scale.

ORIGINAL FINDINGS

It is a well-established finding that, unlike written language, spoken language does not have any clear boundaries between words; spoken language is a continuous stream of sound rather than individual words with silences between them. This lack of segmentation between linguistic units presents a problem for young children learning language, who must be able to pick out individual units from the continuous speech streams that they hear. One proposed method of how children are able to solve this problem is that they are attentive to the statistical regularities of the world around them. For example, in the phrase "pretty baby", children are more likely to hear the sounds pre and ty heard together during the entirety of the lexical input around them than they are to hear the sounds ty and ba together. In an artificial grammar learning study with adult participants, Saffran, Newport, and Aslin found that participants were able to locate word boundaries based only on transitional probabilities, suggesting that adults are capable of using statistical regularities in a language-learning task. This is a robust finding that has been widely replicated.

To determine if young children have these same abilities Saffran Aslin and Newport exposed 8-month-old infants to an artificial grammar. The grammar was composed of four words, each composed of three nonsense syllables. During the experiment, infants heard a continuous speech stream of these words. The speech was presented in a monotone with no cues (such as pauses, intonation, etc.) to word boundaries other than the statistical probabilities. Within a word, the transitional probability of two syllable pairs was 1.0: in the word bidaku, for example, the probability of hearing the syllable da immediately after the syllable bi was 100%. Between words, however, the transitional probability of hearing a syllable pair was much lower: After any given word (e.g., bidaku) was presented, one of three words could follow (in this case, padoti, golabu, or tupiro), so the likelihood of hearing any given syllable after ku was only 33%.

To determine if infants were picking up on the statistical information, each infant was presented with multiple presentations of either a word from the artificial grammar or a nonword made up of the same syllables but presented in a random order. Infants who were presented with nonwords during the test phase listened significantly longer to these words than infants who were presented with words from the artificial grammar, showing a novelty preference for these new nonwords. However, the implementation of the test could also be due to infants learning serial-order information and not to actually learning transitional probabilities between words. That is, at test, infants heard strings such as dapiku and tilado that were never presented during learning; they could simply have learned that the syllable ku never followed the syllable pi.

To look more closely at this issue, Saffran Aslin and Newport conducted another study in which infants underwent the same training with the artificial grammar but then were presented with either words or part-words rather than words or nonwords. The part-words were syllable sequences composed of the last syllable from one word and the first two syllables from another (such as kupado). Because the part-words had been heard during the time when children were listening to the artificial grammar, preferential listening to these part-words would indicate that children were learning not only serial-order information, but also the statistical likelihood of hearing particular syllable sequences. Again, infants showed greater listening times to the novel (part-) words, indicating that 8-month-old infants were able to extract these statistical regularities from a continuous speech stream.

FURTHER RESEARCH

This result has been the impetus for much more research on the role of statistical learning in lexical acquisition and other areas (see). In a follow-up to the original report, Aslin, Saffran, and Newport found that even when words and part words occurred equally often in the speech stream, but with different transitional probabilities between syllables of words and part words, infants were still able to detect the statistical regularities and still preferred to listen to the novel part-words over the familiarized

words. This finding provides stronger evidence that infants are able to pick up transitional probabilities from the speech they hear, rather than just being aware of frequencies of individual syllable sequences.

Another follow-up study examined the extent to which the statistical information learned during this type of artificial grammar learning feeds into knowledge that infants may already have about their native language. Infants preferred to listen to words over part-words, whereas there was no significant difference in the nonsense frame condition. This finding suggests that even pre-linguistic infants are able to integrate the statistical cues they learn in a laboratory into their previously acquired knowledge of a language. In other words, once infants have acquired some linguistic knowledge, they incorporate newly acquired information into that previously acquired learning.

A related finding indicates that slightly older infants can acquire both lexical and grammatical regularities from a single set of input, suggesting that they are able to use outputs of one type of statistical learning (cues that lead to the discovery of word boundaries) as input to a second type (cues that lead to the discovery of syntactical regularities. At test, 12-month-olds preferred to listen to sentences that had the same grammatical structure as the artificial language they had been tested on rather than sentences that had a different (ungrammatical) structure. Because learning grammatical regularities requires infants to be able to determine boundaries between individual words, this indicates that infants who are still quite young are able to acquire multiple levels of language knowledge (both lexical and syntactical) simultaneously, indicating that statistical learning is a powerful mechanism at play in language learning.

Despite the large role that statistical learning appears to play in lexical acquisition, it is likely not the only mechanism by which infants learn to segment words. Statistical learning studies are generally conducted with artificial grammars that have no cues to word boundary information other than transitional probabilities between words. Real speech, though, has many different types of cues to word boundaries, including prosodic and phonotactic information.

Together, the findings from these studies of statistical learning in language acquisition indicate that statistical properties of the language are a strong cue in helping infants learn their first language.

PHONOLOGICAL ACQUISITION

There is much evidence that statistical learning is an important component of both discovering which phonemes are important for a given language and which contrasts within phonemes are important. Having this knowledge is important for aspects of both speech perception and speech production.

DISTRIBUTIONAL LEARNING

Since the discovery of infants' statistical learning abilities in word learning, the same general mechanism has also been studied in other facets of language learning. For example, it is well-established that infants can discriminate between phonemes of many different languages but eventually become unable to discriminate between phonemes that do not appear in their native language; however, it was not clear how this decrease in discriminatory ability came about. Maye et al. suggested that the mechanism responsible might be a statistical learning mechanism in which infants track the distributional regularities of the sounds in their native language. To test this idea, Maye et al. exposed 6- and 8-month-old infants to a continuum of speech sounds that varied on the degree to which they were voiced. The distribution that the infants heard was either bimodal, with sounds from both ends of the voicing continuum heard most often, or unimodal, with sounds from the middle of the distribution heard most often. The results indicated that infants from both age groups were sensitive to the distribution of phonemes. At test, infants heard either non-alternating (repeated exemplars of tokens 3 or 6 from an 8-token continuum) or alternating (exemplars of tokens 1 and 8) exposures to specific phonemes on the continuum. Infants exposed to the bimodal distribution listened longer to the alternating trials than the non-alternating trials while there was no difference in listening times for infants exposed to the unimodal distribution. This finding indicates that infants exposed the bimodal distribution were better able to discriminate sounds from the two ends of the distribution than were infants in the unimodal condition, regardless of age. This type of statistical learning differs from that used in lexical acquisition, as it requires infants to track frequencies rather than transitional

probabilities, and has been named "distributional learning".

Distributional learning has also been found to help infants contrast two phonemes that they initially have difficulty in discriminating between. Maye, Weiss, and Aslin found that infants who were exposed to a bimodal distribution of a non-native contrast that was initially difficult to discriminate were better able to discriminate the contrast than infants exposed to a unimodal distribution of the same contrast. Maye et al. also found that infants were able to abstract features of a contrast (i.e., voicing onset time) and generalize that feature to the same type of contrast at a different place of articulation, a finding that has not been found in adults.

In a review of the role of distributional learning on phonological acquisition, Werker et al. note that distributional learning cannot be the only mechanism by which phonetic categories are acquired. However, it does seem clear that this type of statistical learning mechanism can play a role in this skill, although research is ongoing.

PERCEPTUAL MAGNET EFFECT

A related finding regarding statistical cues to phonological acquisition is a phenomenon known as the perceptual magnet effect. In this effect, a prototypical phoneme of a person's native language acts as a "magnet" for similar phonemes, which are perceived as belonging to the same category as the prototypical phoneme. In the original test of this effect, adult participants were asked to indicate if a given exemplar of a particular phoneme differed from a referent phoneme. If the referent phoneme is a non-prototypical phoneme for that language, both adults and 6-month-old infants show less generalization to other sounds than they do for prototypical phonemes, even if the subjective distance between the sounds is the same. That is, adults and infants are both more likely to notice that a particular phoneme differs from the referent phoneme if that referent phoneme is a non-prototypical exemplar than if it is a prototypical exemplar. The prototypes themselves are apparently discovered through a distributional learning process, in which infants are sensitive to the frequencies with which certain sounds occur and treat those that occur most often as the prototypical phonemes of their language.

SYNTACTICAL ACQUISITION

A statistical learning device has also been proposed as a component of syntactical acquisition for young children. Early evidence for this mechanism came largely from studies of computer modeling or analyses of natural language corpora. These early studies focused largely on distributional information specifically rather than statistical learning mechanisms generally. Specifically, in these early papers it was proposed that children created templates of possible sentence structures involving unnamed categories of word types (i.e., nouns or verbs, although children would not put these labels on their categories). Children were thought to learn which words belonged to the same categories by tracking the similar contexts in which words of the same category appeared.

Later studies expanded these results by looking at the actual behavior of children or adults who had been exposed to artificial grammars. These later studies also considered the role of statistical learning more broadly than the earlier studies, placing their results in the context of the statistical learning mechanisms thought to be involved with other aspects of language learning, such as lexical acquisition.

EXPERIMENTAL RESULTS

Evidence from a series of four experiments conducted by Gomez and Gerken suggests that children are able to generalize grammatical structures with less than two minutes of exposure to an artificial grammar. In the first experiment, 11-12 month-old infants were trained on an artificial grammar composed of nonsense words with a set grammatical structure. At test, infants heard both novel grammatical and ungrammatical sentences. Infants oriented longer toward the grammatical sentences, in line with previous research that suggests that infants generally orient for a longer amount of time to natural instances of language rather than altered instances of language e.g.,. (This familiarity preference differs from the novelty preference generally found in word-learning studies, due to the

differences between lexical acquisition and syntactical acquisition.) This finding indicates that young children are sensitive to the grammatical structure of language even after minimal exposure. Gomez and Gerken also found that this sensitivity is evident when ungrammatical transitions are located in the middle of the sentence (unlike in the first experiment, in which all the errors occurred at the beginning and end of the sentences), that the results could not be due to an innate preference for the grammatical sentences caused by something other than grammar, and that children are able to generalize the grammatical rules to new vocabulary.

Together these studies suggest that infants are able to extract a substantial amount of syntactic knowledge even from limited exposure to a language. Children apparently detected grammatical anomalies whether the grammatical violation in the test sentences occurred at the end or in the middle of the sentence. Additionally, even when the individual words of the grammar were changed, infants were still able to discriminate between grammatical and ungrammatical strings during the test phase. This generalization indicates that infants were not learning vocabulary-specific grammatical structures, but abstracting the general rules of that grammar and applying those rules to novel vocabulary. Furthermore, in all four experiments, the test of grammatical structures occurred five minutes after the initial exposure to the artificial grammar had ended, suggesting that the infants were able to maintain the grammatical abstractions they had learned even after a short delay.

In a similar study, Saffran found that adults and older children (first- and second grade children) were also sensitive to syntactical information after exposure to an artificial language which had no cues to phrase structure other than the statistical regularities that were present. Both adults and children were able to pick out sentences that were ungrammatical at a rate greater than chance, even under an "incidental" exposure condition in which participants' primary goal was to complete a different task while hearing the language.

Although the number of studies dealing with statistical learning of syntactical information is limited, the available evidence does indicate that the statistical learning mechanisms are likely a contributing factor to children's ability to learn their language.

STATISTICAL LEARNING IN BILINGUALISM

Much of the early work using statistical learning paradigms focused on the ability for children or adults to learn a single language, consistent with the process of language acquisition for monolingual speakers or learners. However, it is estimated that approximately 60-75% of people in the world are bilingual. More recently, researchers have begun looking at the role of statistical learning for those who speak more than one language. Although there are no reviews on this topic yet, Weiss, Gerfen, and Mitchel examined how hearing input from multiple artificial languages simultaneously can affect the ability to learn either or both languages. Over four experiments, Weiss et al. found that, after exposure to two artificial languages, adult learners are capable of determining word boundaries in both languages when each language is spoken by a different speaker. However, when the two languages were spoken by the same speaker, participants were able learn both languages only when they were "congruent"—when the word boundaries of one language matched the word boundaries of the other. When the languages were incongruent—a syllable that appeared in the middle of a word in one language appeared at the end of the word in the other language—and spoken by a single speaker, participants were able to learn, at best, one of the two languages. A final experiment showed that the inability to learn incongruent languages spoken in the same voice was not due to syllable overlap between the languages but due to differing word boundaries.

Similar work replicates the finding that learners are able to learn two sets of statistical representations when an additional cue is present (two different male voices in this case). In their paradigm, the two languages were presented consecutively, rather than interleaved as in Weiss et al.'s paradigm, and participants did learn the first artificial language to which they had been exposed better than the second, although participants' performance was above chance for both languages.

While statistical learning improves and strengthens multilingualism, it appears that the inverse is not true. In a study by Yim and Rudoy it was found that both monolingual and bilingual children perform statistical learning tasks equally well.

Antovich and Graf Estes found that 14-month-old bilingual children are better than monolinguals at segmenting two different artificial languages using transitional probability cues. They suggest that a bilingual environment in early childhood trains children to rely on statistical regularities to segment the speech flow and access two lexical systems.

LIMITATIONS ON STATISTICAL LEARNING

WORD-REFERENT MAPPING

A statistical learning mechanism has also been proposed for learning the meaning of words. Specifically, Yu and Smith conducted a pair of studies in which adults were exposed to pictures of objects and heard nonsense words. Each nonsense word was paired with a particular object. There were 18 total word-referent pairs, and each participant was presented with either 2, 3, or 4 objects at a time, depending on the condition, and heard the nonsense word associated with one of those objects. Each word-referent pair was presented 6 times over the course of the training trials; after the completion of the training trials, participants completed a forced-alternative test in which they were asked to choose the correct referent that matched a nonsense word they were given. Participants were able to choose the correct item more often than would happen by chance, indicating, according to the authors, that they were using statistical learning mechanisms to track co-occurrence probabilities across training trials.

An alternative hypothesis is that learners in this type of task may be using a "propose-but-verify" mechanism rather than a statistical learning mechanism. Medina et al. and Trueswell et al. argue that, because Yu and Smith only tracked knowledge at the end of the training, rather than tracking knowledge on a trial-by-trial basis, it is impossible to know if participants were truly updating statistical probabilities of co-occurrence (and therefore maintaining multiple hypotheses simultaneously), or if, instead, they were forming a single hypothesis and checking it on the next trial. For example, if a participant is presented with a picture of a dog and a picture of a shoe, and hears the nonsense word vash she might hypothesize that vash refers to the dog. On a future trial, she may see a picture of a shoe and a picture of a door and again hear the word vash. If statistical learning is the mechanism by which word-referent mappings are learned, then the participant would be more likely to select the picture of the shoe than the door, as shoe would have appeared in conjunction with the word vash 100% of the time. However, if participants are simply forming a single hypothesis, they may fail to remember the context of the previous presentation of vash (especially if, as in the experimental conditions, there are multiple trials with other words in between the two presentations of vash) and therefore be at chance in this second trial. According to this proposed mechanism of word learning, if the participant had correctly guessed that vash referred to the shoe in the first trial, her hypothesis would be confirmed in the subsequent trial.

To distinguish between these two possibilities, Trueswell et al. conducted a series of experiments similar to those conducted by Yu and Smith except that participants were asked to indicate their choice of the word-referent mapping on each trial, and only a single object name was presented on each trial (with varying numbers of objects). Participants would therefore have been at chance when they are forced to make a choice in their first trial. The results from the subsequent trials indicate that participants were not using a statistical learning mechanism in these experiments, but instead were using a propose-and-verify mechanism, holding only one potential hypothesis in mind at a time. Specifically, if participants had chosen an incorrect word-referent mapping in an initial presentation of a nonsense word (from a display of five possible choices), their likelihood of choosing the correct word-referent mapping in the next trial of that word was still at chance, or 20%. If, though, the participant had chosen the correct word-referent mapping on an initial presentation of a nonsense word, the likelihood of choosing the correct word-referent mapping on the subsequent presentation of that word was approximately 50%. These results were also replicated in a condition where participants were choosing between only two alternatives. These results suggest that participants did not remember the surrounding context of individual presentations and were therefore not using statistical cues to determine the word-referent mappings. Instead, participants make a hypothesis regarding a word-referent mapping and, on the next presentation of that word, either confirm or reject the

hypothesis accordingly.

Overall, these results, along with similar results from Medina et al., indicate that word meanings may not be learned through a statistical learning mechanism in these experiments, which ask participants to hypothesize a mapping even on the first occurrence (i.e., not cross-situationally). However, when the propose-but-verify mechanism has been compared to a statistical learning mechanism, the former was unable to reproduce individual learning trajectories nor fit as well as the latter.

NEED FOR SOCIAL INTERACTION

Additionally, statistical learning by itself cannot account even for those aspects of language acquisition for which it has been shown to play a large role. For example, Kuhl, Tsao, and Liu found that young English-learning infants who spent time in a laboratory session with a native Mandarin speaker were able to distinguish between phonemes that occur in Mandarin but not in English, unlike infants who were in a control condition. Infants in this control condition came to the lab as often as infants in the experimental condition, but were exposed only to English; when tested at a later date, they were unable to distinguish the Mandarin phonemes. In a second experiment, the authors presented infants with audio or audiovisual recordings of Mandarin speakers and tested the infants' ability to distinguish between the Mandarin phonemes. In this condition, infants failed to distinguish the foreign language phonemes. This finding indicates that social interaction is a necessary component of language learning and that, even if infants are presented with the raw data of hearing a language, they are unable to take advantage of the statistical cues present in that data if they are not also experiencing the social interaction.

DOMAIN GENERALITY

Although the phenomenon of statistical learning was first discovered in the context of language acquisition and there is much evidence of its role in that purpose, work since the original discovery has suggested that statistical learning may be a domain general skill and is likely not unique to humans. For example, Saffran, Johnson, Aslin, and Newport found that both adults and infants were able to learn statistical probabilities of "words" created by playing different musical tones (i.e., participants heard the musical notes D, E, and F presented together during training and were able to recognize those notes as a unit at test as compared to three notes that had not been presented together). In non-auditory domains, there is evidence that humans are able to learn statistical visual information whether that information is presented across space, e.g., or time, e.g.,. Evidence of statistical learning has also been found in other primates, e.g., and some limited statistical learning abilities have been found even in non-primates like rats. Together these findings suggest that statistical learning may be a generalized learning mechanism that happens to be utilized in language acquisition, rather than a mechanism that is unique to the human infant's ability to learn his or her language(s).

Further evidence for domain general statistical learning was suggested in a study run through the University of Cornell Department of Psychology concerning visual statistical learning in infancy. Researchers in this study questioned whether domain generality of statistical learning in infancy would be seen using visual information. After first viewing images in statistically predictable patterns, infants were then exposed to the same familiar patterns in addition to novel sequences of the same identical stimulus components. Interest in the visuals was measured by the amount of time the child looked at the stimuli in which the researchers named "looking time". All ages of infant participants showed more interest in the novel sequence relative to the familiar sequence. In demonstrating a preference for the novel sequences (which violated the transitional probability that defined the grouping of the original stimuli) the results of the study support the likelihood of domain general statistical learning in infancy.

Data mining is the process of extracting and finding patterns in massive data sets involving methods at the intersection of machine learning, statistics, and database systems. Data mining is an interdisciplinary subfield of computer science and statistics with an overall goal of extracting information (with intelligent methods) from a data set and transforming the information into a comprehensible structure for further use. Data mining is the analysis step of the "knowledge discovery in databases" process, or KDD. Aside from the raw analysis step, it also involves database and data management

aspects, data pre-processing, model and inference considerations, interestingness metrics, complexity considerations, post-processing of discovered structures, visualization, and online updating.

The term "data mining" is a misnomer because the goal is the extraction of patterns and knowledge from large amounts of data, not the extraction (mining) of data itself. It also is a buzzword and is frequently applied to any form of large-scale data or information processing (collection, extraction, warehousing, analysis, and statistics) as well as any application of computer decision support systems, including artificial intelligence (e.g., machine learning) and business intelligence. Often the more general terms (large scale) data analysis and analytics—or, when referring to actual methods, artificial intelligence and machine learning—are more appropriate.

The actual data mining task is the semi-automatic or automatic analysis of massive quantities of data to extract previously unknown, interesting patterns such as groups of data records (cluster analysis), unusual records (anomaly detection), and dependencies (association rule mining, sequential pattern mining). This usually involves using database techniques such as spatial indices. These patterns can then be seen as a kind of summary of the input data, and may be used in further analysis or, for example, in machine learning and predictive analytics. For example, the data mining step might identify multiple groups in the data, which can then be used to obtain more accurate prediction results by a decision support system. Neither the data collection, data preparation, nor result interpretation and reporting is part of the data mining step, although they do belong to the overall KDD process as additional steps.

The difference between data analysis and data mining is that data analysis is used to test models and hypotheses on the dataset, e.g., analyzing the effectiveness of a marketing campaign, regardless of the amount of data. In contrast, data mining uses machine learning and statistical models to uncover clandestine or hidden patterns in a large volume of data.

The related terms data dredging, data fishing, and data snooping refer to the use of data mining methods to sample parts of a larger population data set that are (or may be) too small for reliable statistical inferences to be made about the validity of any patterns discovered. These methods can, however, be used in creating new hypotheses to test against the larger data populations.

ETYMOLOGY

In the 1960s, statisticians and economists used terms like data fishing or data dredging to refer to what they considered the bad practice of analyzing data without an a-priori hypothesis. The term "data mining" was used in a similarly critical way by economist Michael Lovell in an article published in the Review of Economic Studies in 1983. Lovell indicates that the practice "masquerades under a variety of aliases, ranging from "experimentation" (positive) to "fishing" or "snooping" (negative).

The term data mining appeared around 1990 in the database community, with generally positive connotations. For a short time in 1980s, the phrase "database mining"™, was used, but since it was trademarked by HNC, a San Diego–based company, to pitch their Database Mining Workstation; researchers consequently turned to data mining. Other terms used include data archaeology, information harvesting, information discovery, knowledge extraction, etc. Gregory Piatetsky-Shapiro coined the term "knowledge discovery in databases" for the first workshop on the same topic (KDD-1989) and this term became more popular in the AI and machine learning communities. However, the term data mining became more popular in the business and press communities. Currently, the terms data mining and knowledge discovery are used interchangeably.

BACKGROUND

The manual extraction of patterns from data has occurred for centuries. Early methods of identifying patterns in data include Bayes' theorem (1700s) and regression analysis (1800s). The proliferation, ubiquity and increasing power of computer technology have dramatically increased data collection, storage, and manipulation ability. As data sets have grown in size and complexity, direct "hands-on" data analysis has increasingly been augmented with indirect, automated data processing, aided by

other discoveries in computer science, specially in the field of machine learning, such as neural networks, cluster analysis, genetic algorithms (1950s), decision trees and decision rules (1960s), and support vector machines (1990s). Data mining is the process of applying these methods with the intention of uncovering hidden patterns. in large data sets. It bridges the gap from applied statistics and artificial intelligence (which usually provide the mathematical background) to database management by exploiting the way data is stored and indexed in databases to execute the actual learning and discovery algorithms more efficiently, allowing such methods to be applied to ever-larger data sets.

PROCESS

The knowledge discovery in databases (KDD) process is commonly defined with the stages:

It exists, however, in many variations on this theme, such as the Cross-industry standard process for data mining (CRISP-DM) which defines six phases:

or a simplified process such as (1) Pre-processing, (2) Data Mining, and (3) Results Validation.

Polls conducted in 2002, 2004, 2007 and 2014 show that the CRISP-DM methodology is the leading methodology used by data miners.

The only other data mining standard named in these polls was SEMMA. However, 3–4 times as many people reported using CRISP-DM. Several teams of researchers have published reviews of data mining process models, and Azevedo and Santos conducted a comparison of CRISP-DM and SEMMA in 2008.

PRE-PROCESSING

Before data mining algorithms can be used, a target data set must be assembled. As data mining can only uncover patterns actually present in the data, the target data set must be large enough to contain these patterns while remaining concise enough to be mined within an acceptable time limit. A common source for data is a data mart or data warehouse. Pre-processing is essential to analyze the multivariate data sets before data mining. The target set is then cleaned. Data cleaning removes the observations containing noise and those with missing data.

DATA MINING

Data mining involves six common classes of tasks:

RESULTS VALIDATION

Data mining can unintentionally be misused, producing results that appear to be significant but which do not actually predict future behavior and cannot be reproduced on a new sample of data, therefore bearing little use. This is sometimes caused by investigating too many hypotheses and not performing proper statistical hypothesis testing. A simple version of this problem in machine learning is known as overfitting, but the same problem can arise at different phases of the process and thus a train/test split—when applicable at all—may not be sufficient to prevent this from happening.

The final step of knowledge discovery from data is to verify that the patterns produced by the data mining algorithms occur in the wider data set. Not all patterns found by the algorithms are necessarily valid. It is common for data mining algorithms to find patterns in the training set which are not present in the general data set. This is called overfitting. To overcome this, the evaluation uses a test set of data on which the data mining algorithm was not trained. The learned patterns are applied to this test set, and the resulting output is compared to the desired output. For example, a data mining algorithm trying to distinguish "spam" from "legitimate" e-mails would be trained on a training set of sample e-mails. Once trained, the learned patterns would be applied to the test set of e-mails on which it had not been trained. The accuracy of the patterns can then be measured from how many e-mails they correctly classify. Several statistical methods may be used to evaluate the algorithm, such as ROC curves.

If the learned patterns do not meet the desired standards, it is necessary to re-evaluate and change the pre-processing and data mining steps. If the learned patterns do meet the desired standards, then the final step is to interpret the learned patterns and turn them into knowledge.

RESEARCH

The premier professional body in the field is the Association for Computing Machinery's (ACM) Special Interest Group (SIG) on Knowledge Discovery and Data Mining (SIGKDD). Since 1989, this ACM SIG has hosted an annual international conference and published its proceedings, and since 1999 it has published a biannual academic journal titled "SIGKDD Explorations".

Computer science conferences on data mining include:

Data mining topics are also present in many data management/database conferences such as the ICDE Conference, SIGMOD Conference and International Conference on Very Large Data Bases.

STANDARDS

There have been some efforts to define standards for the data mining process, for example, the 1999 European Cross Industry Standard Process for Data Mining (CRISP-DM 1.0) and the 2004 Java Data Mining standard (JDM 1.0). Development on successors to these processes (CRISP-DM 2.0 and JDM 2.0) was active in 2006 but has stalled since. JDM 2.0 was withdrawn without reaching a final draft.

For exchanging the extracted models—in particular for use in predictive analytics—the key standard is the Predictive Model Markup Language (PMML), which is an XML-based language developed by the Data Mining Group (DMG) and supported as exchange format by many data mining applications. As the name suggests, it only covers prediction models, a particular data mining task of high importance to business applications. However, extensions to cover (for example) subspace clustering have been proposed independently of the DMG.

NOTABLE USES

Data mining is used wherever there is digital data available. Notable examples of data mining can be found throughout business, medicine, science, finance, construction, and surveillance.

PRIVACY CONCERNS AND ETHICS

While the term "data mining" itself may have no ethical implications, it is often associated with the mining of information in relation to user behavior (ethical and otherwise).

The ways in which data mining can be used can in some cases and contexts raise questions regarding privacy, legality, and ethics. In particular, data mining government or commercial data sets for national security or law enforcement purposes, such as in the Total Information Awareness Program or in ADVISE, has raised privacy concerns.

Data mining requires data preparation which uncovers information or patterns which compromise confidentiality and privacy obligations. A common way for this to occur is through data aggregation. Data aggregation involves combining data together (possibly from various sources) in a way that facilitates analysis (but that also might make identification of private, individual-level data deducible or otherwise apparent). This is not data mining per se, but a result of the preparation of data before—and for the purposes of—the analysis. The threat to an individual's privacy comes into play when the data, once compiled, cause the data miner, or anyone who has access to the newly compiled data set, to be able to identify specific individuals, especially when the data were originally anonymous.

Data may also be modified so as to become anonymous, so that individuals may not readily be identified. However, even "anonymized" data sets can potentially contain enough information to allow identification of individuals, as occurred when journalists were able to find several individuals based on a set of search histories that were inadvertently released by AOL.

The inadvertent revelation of personally identifiable information leading to the provider violates Fair Information Practices. This indiscretion can cause financial, emotional, or bodily harm to the indicated individual. In one instance of privacy violation, the patrons of Walgreens filed a lawsuit against the company in 2011 for selling prescription information to data mining companies who in turn provided the data to pharmaceutical companies.

SITUATION IN EUROPE

Europe has rather strong privacy laws, and efforts are underway to further strengthen the rights of the consumers. However, the U.S.–E.U. Safe Harbor Principles, developed between 1998 and 2000, currently effectively expose European users to privacy exploitation by U.S. companies. As a consequence of Edward Snowden's global surveillance disclosure, there has been increased discussion to revoke this agreement, as in particular the data will be fully exposed to the National Security Agency, and attempts to reach an agreement with the United States have failed.

In the United Kingdom in particular there have been cases of corporations using data mining as a way to target certain groups of customers forcing them to pay unfairly high prices. These groups tend to be people of lower socio-economic status who are not savvy to the ways they can be exploited in digital market places.

SITUATION IN THE UNITED STATES

In the United States, privacy concerns have been addressed by the US Congress via the passage of regulatory controls such as the Health Insurance Portability and Accountability Act (HIPAA). The HIPAA requires individuals to give their "informed consent" regarding information they provide and its intended present and future uses. According to an article in Biotech Business Week, "' n practice, HIPAA may not offer any greater protection than the longstanding regulations in the research arena,' says the AAHC. More importantly, the rule's goal of protection through informed consent is approaching a level of incomprehensibility to average individuals." This underscores the necessity for data anonymity in data aggregation and mining practices.

U.S. information privacy legislation such as HIPAA and the Family Educational Rights and Privacy Act (FERPA) applies only to the specific areas that each such law addresses. The use of data mining by the majority of businesses in the U.S. is not controlled by any legislation.

COPYRIGHT LAW

SITUATION IN EUROPE

Even if there is no copyright in a dataset, the European Union recognises a Database right, so data mining becomes subject to intellectual property owners' rights that are protected by the Database Directive. Under European copyright database laws, the mining of in-copyright works (such as by web mining) without the permission of the copyright owner is permitted under Articles 3 and 4 of the 2019 Directive on Copyright in the Digital Single Market. A specific TDM exception for scientific research is described in article 3, whereas a more general exception described in article 4 only applies if the copyright holder has not opted out.

The European Commission facilitated stakeholder discussion on text and data mining in 2013, under the title of Licences for Europe. The focus on the solution to this legal issue, such as licensing rather than limitations and exceptions, led to representatives of universities, researchers, libraries, civil society groups and open access publishers to leave the stakeholder dialogue in May 2013.

On the recommendation of the Hargreaves review, this led to the UK government to amend its copyright law in 2014 to allow content mining as a limitation and exception. The UK was the second country in the world to do so after Japan, which introduced an exception in 2009 for data mining. However, due to the restriction of the Information Society Directive (2001), the UK exception only allows content mining for non-commercial purposes. UK copyright law also does not allow this provision to be overridden by contractual terms and conditions.

Since 2020 also Switzerland has been regulating data mining by allowing it in the research field under certain conditions laid down by art. 24d of the Swiss Copyright Act. This new article entered into force on 1 April 2020.

SITUATION IN THE UNITED STATES

US copyright law, and in particular its provision for fair use, upholds the legality of content mining in America, and other fair use countries such as Israel, Taiwan and South Korea. As content mining is transformative, that is it does not supplant the original work, it is viewed as being lawful under fair use. For example, as part of the Google Book settlement the presiding judge on the case ruled that Google's digitization project of in-copyright books was lawful, in part because of the transformative uses that the digitization project displayed—one being text and data mining.

SOFTWARE

FREE OPEN-SOURCE DATA MINING SOFTWARE AND APPLICATIONS

The following applications are available under free/open-source licenses. Public access to application source code is also available.

PROPRIETARY DATA-MINING SOFTWARE AND APPLICATIONS

The following applications are available under proprietary licenses.

In machine learning, supervised learning (SL) is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on example input-output pairs. This process involves training a statistical model using labeled data, meaning each piece of input data is provided with the correct output. For instance, if you want a model to identify cats in images, supervised learning would involve feeding it many images of cats (inputs) that are explicitly labeled "cat" (outputs).

The goal of supervised learning is for the trained model to accurately predict the output for new, unseen data. This requires the algorithm to effectively generalize from the training examples, a quality measured by its generalization error. Supervised learning is commonly used for tasks like classification (predicting a category, e.g., spam or not spam) and regression (predicting a continuous value, e.g., house prices).

STEPS TO FOLLOW

To solve a given problem of supervised learning, the following steps must be performed:

ALGORITHM CHOICE

A wide range of supervised learning algorithms are available, each with its strengths and weaknesses. There is no single learning algorithm that works best on all supervised learning problems (see the No free lunch theorem).

There are four major issues to consider in supervised learning:

BIAS–VARIANCE TRADEOFF

A first issue is the tradeoff between bias and variance. Imagine that we have available several different, but equally good, training data sets. A learning algorithm is biased for a particular input x {\displaystyle x} if, when trained on each of these data sets, it is systematically incorrect when predicting the correct output for x {\displaystyle x} . A learning algorithm has high variance for a particular input x {\displaystyle x} if it predicts different output values when trained on different training sets. The prediction error of a learned classifier is related to the sum of the bias and the variance of the learning algorithm. Generally, there is a tradeoff between bias and variance. A learning algorithm with low bias must be "flexible" so that it can fit the data well. But if the learning algorithm is too flexible, it will fit each training data set differently, and hence have high variance. A key aspect of many supervised learning methods is that they are able to adjust this tradeoff between bias and variance (either automatically or by providing a bias/variance parameter that the user can adjust).

FUNCTION COMPLEXITY AND AMOUNT OF TRAINING DATA

The second issue is of the amount of training data available relative to the complexity of the "true" function (classifier or regression function). If the true function is simple, then an "inflexible" learning algorithm with high bias and low variance will be able to learn it from a small amount of data. But if the true function is highly complex (e.g., because it involves complex interactions among many different input features and behaves differently in different parts of the input space), then the function will only be able to learn with a large amount of training data paired with a "flexible" learning algorithm with low bias and high variance.

DIMENSIONALITY OF THE INPUT SPACE

A third issue is the dimensionality of the input space. If the input feature vectors have large dimensions, learning the function can be difficult even if the true function only depends on a small number of those features. This is because the many "extra" dimensions can confuse the learning algorithm and cause it to have high variance. Hence, input data of large dimensions typically requires tuning the classifier to have low variance and high bias. In practice, if the engineer can manually remove irrelevant features from the input data, it will likely improve the accuracy of the learned function. In addition, there are many algorithms for feature selection that seek to identify the relevant features and discard the irrelevant ones. This is an instance of the more general strategy of dimensionality reduction, which seeks to map the input data into a lower-dimensional space prior to running the supervised learning algorithm.

NOISE IN THE OUTPUT VALUES

A fourth issue is the degree of noise in the desired output values (the supervisory target variables). If the desired output values are often incorrect (because of human error or sensor errors), then the learning algorithm should not attempt to find a function that exactly matches the training examples. Attempting to fit the data too carefully leads to overfitting. You can overfit even when there are no measurement errors (stochastic noise) if the function you are trying to learn is too complex for your learning model. In such a situation, the part of the target function that cannot be modeled "corrupts" your training data – this phenomenon has been called deterministic noise. When either type of noise is present, it is better to go with a higher bias, lower variance estimator.

In practice, there are several approaches to alleviate noise in the output values such as early stopping to prevent overfitting as well as detecting and removing the noisy training examples prior to training the supervised learning algorithm. There are several algorithms that identify noisy training examples and removing the suspected noisy training examples prior to training has decreased generalization error with statistical significance.

OTHER FACTORS TO CONSIDER

Other factors to consider when choosing and applying a learning algorithm include the following:

When considering a new application, the engineer can compare multiple learning algorithms and experimentally determine which one works best on the problem at hand (see cross-validation). Tuning the performance of a learning algorithm can be very time-consuming. Given fixed resources, it is often better to spend more time collecting additional training data and more informative features than it is to spend extra time tuning the learning algorithms.

ALGORITHMS

The most widely used learning algorithms are:

HOW SUPERVISED LEARNING ALGORITHMS WORK

Given a set of N {\displaystyle N} training examples of the form { (x 1 , y 1) , . . . , (x N , y N) } {\displaystyle \{(x_{1},y_{1}),...,(x_{N},\;y_{N})\}} such that x i {\displaystyle x_{i}} is the feature vector of the i {\displaystyle i} -th example and y i {\displaystyle y_{i}} is its label (i.e., class), a learning algorithm seeks a function g : X → Y {\displaystyle g:X\to Y} , where X {\displaystyle X} is the input space and Y {\displaystyle Y} is the output space. The function g {\displaystyle g} is an element of some space of possible functions G {\displaystyle G} , usually called the hypothesis space. It is sometimes convenient to represent g {\displaystyle g} using a scoring function f : X × Y → R {\displaystyle f:X\times Y\to \mathbb {R} } such that g {\displaystyle g} is defined as returning the y {\displaystyle y} value that gives the highest score: g (x) = arg ■ max y f (x , y) {\displaystyle g(x)={\underset {y}{\arg \max }}\;f(x,y)} . Let F {\displaystyle F} denote the space of scoring functions.

Although G {\displaystyle G} and F {\displaystyle F} can be any space of functions, many learning algorithms are probabilistic models where g {\displaystyle g} takes the form of a conditional probability model g (x) = arg ■ max y P (y | x) {\displaystyle g(x)={\underset {y}{\arg \max }}\;P(y|x)} , or f {\displaystyle f} takes the form of a joint probability model f (x , y) = P (x , y) {\displaystyle f(x,y)=P(x,y)} . For example, naive Bayes and linear discriminant analysis are joint probability models, whereas logistic regression is a conditional probability model.

There are two basic approaches to choosing f {\displaystyle f} or g {\displaystyle g} : empirical risk minimization and structural risk minimization. Empirical risk minimization seeks the function that best fits the training data. Structural risk minimization includes a penalty function that controls the bias/variance tradeoff.

In both cases, it is assumed that the training set consists of a sample of independent and identically distributed pairs, (x i , y i) {\displaystyle (x_{i},\;y_{i})} . In order to measure how well a function fits the training data, a loss function L : Y × Y → R ≥ 0 {\displaystyle L:Y\times Y\to \mathbb {R} ^{\geq 0}} is defined. For training example (x i , y i) {\displaystyle (x_{i},\;y_{i})} , the loss of predicting the value y ^ {\displaystyle {\hat {y}}} is L (y i , y ^) {\displaystyle L(y_{i},{\hat {y}})} .

The risk R (g) {\displaystyle R(g)} of function g {\displaystyle g} is defined as the expected loss of g {\displaystyle g} . This can be estimated from the training data as

EMPIRICAL RISK MINIMIZATION

In empirical risk minimization, the supervised learning algorithm seeks the function g {\displaystyle g} that minimizes R (g) {\displaystyle R(g)} . Hence, a supervised learning algorithm can be constructed by applying an optimization algorithm to find g {\displaystyle g} .

When g {\displaystyle g} is a conditional probability distribution P (y | x) {\displaystyle P(y|x)} and the loss function is the negative log likelihood: L (y , y ^) = – log ■ P (y | x) {\displaystyle L(y,{\hat {y}})=-\log P(y|x)} , then empirical risk minimization is equivalent to maximum likelihood estimation.

When G {\displaystyle G} contains many candidate functions or the training set is not sufficiently large, empirical risk minimization leads to high variance and poor generalization. The learning algorithm is able to memorize the training examples without generalizing well (overfitting).

STRUCTURAL RISK MINIMIZATION

Structural risk minimization seeks to prevent overfitting by incorporating a regularization penalty into the optimization. The regularization penalty can be viewed as implementing a form of Occam's razor that prefers simpler functions over more complex ones.

A wide variety of penalties have been employed that correspond to different definitions of complexity. For example, consider the case where the function g {\displaystyle g} is a linear function of the form

A popular regularization penalty is ∑ j β j 2 {\displaystyle \sum _{j}\beta _{j}^{2}} , which is the squared Euclidean norm of the weights, also known as the L 2 {\displaystyle L_{2}} norm. Other norms include the L 1 {\displaystyle L_{1}} norm, ∑ j | β j | {\displaystyle \sum _{j}|\beta _{j}|} , and the L 0 {\displaystyle L_{0}} "norm", which is the number of non-zero β j {\displaystyle \beta _{j}} s. The penalty will be denoted by C (g) {\displaystyle C(g)} .

The supervised learning optimization problem is to find the function g {\displaystyle g} that minimizes

The parameter λ {\displaystyle \lambda } controls the bias-variance tradeoff. When λ = 0 {\displaystyle \lambda =0} , this gives empirical risk minimization with low bias and high variance. When λ {\displaystyle \lambda } is large, the learning algorithm will have high bias and low variance. The value of λ {\displaystyle \lambda } can be chosen empirically via cross-validation.

The complexity penalty has a Bayesian interpretation as the negative log prior probability of g {\displaystyle g} , – log ■ P (g) {\displaystyle -\log P(g)} , in which case J (g) {\displaystyle J(g)} is the posterior probability of g {\displaystyle g} .

GENERATIVE TRAINING

The training methods described above are discriminative training methods, because they seek to find a function g {\displaystyle g} that discriminates well between the different output values (see discriminative model). For the special case where f (x , y) = P (x , y) {\displaystyle f(x,y)=P(x,y)} is a joint probability distribution and the loss function is the negative log likelihood – ∑ i log ■ P (x i , y i) , {\displaystyle -\sum _{i}\log P(x_{i},y_{i}),} a risk minimization algorithm is said to perform generative training, because f {\displaystyle f} can be regarded as a generative model that explains how the data were generated. Generative training algorithms are often simpler and more computationally efficient than discriminative training algorithms. In some cases, the solution can be computed in closed form as in naive Bayes and linear discriminant analysis.

GENERALIZATIONS

There are several ways in which the standard supervised learning problem can be generalized:

APPROACHES AND ALGORITHMS

APPLICATIONS

GENERAL ISSUES

Unsupervised learning is a framework in machine learning where, in contrast to supervised learning, algorithms learn patterns exclusively from unlabeled data. Other frameworks in the spectrum of supervisions include weak- or semi-supervision, where a small portion of the data is tagged, and self-supervision. Some researchers consider self-supervised learning a form of unsupervised learning.

Conceptually, unsupervised learning divides into the aspects of data, training, algorithm, and downstream applications. Typically, the dataset is harvested cheaply "in the wild", such as massive text corpus obtained by web crawling, with only minor filtering (such as Common Crawl). This compares favorably to supervised learning, where the dataset (such as the ImageNet1000) is typically constructed manually, which is much more expensive.

There are algorithms designed specifically for unsupervised learning, such as clustering algorithms like k-means, dimensionality reduction techniques like principal component analysis (PCA), Boltzmann machine learning, and autoencoders. After the rise of deep learning, most large-scale unsupervised learning has been done by training general-purpose neural network architectures by gradient descent, adapted to performing unsupervised learning by designing an appropriate training procedure.

Sometimes a trained model can be used as-is, but more often they are modified for downstream applications. For example, the generative pretraining method trains a model to generate a textual dataset, before finetuning it for other applications, such as text classification. As another example, autoencoders are trained to produce good features, which can then be used as a module for other models, such as in a latent diffusion model.

TASKS

Tasks are often categorized as discriminative (recognition) or generative (imagination). Often but not always, discriminative tasks use supervised methods and generative tasks use unsupervised (see Venn diagram); however, the separation is very hazy. For example, object recognition favors supervised learning but unsupervised learning can also cluster objects into groups. Furthermore, as progress marches onward, some tasks employ both methods, and some tasks swing from one to another. For example, image recognition started off as heavily supervised, but became hybrid by employing unsupervised pre-training, and then moved towards supervision again with the advent of dropout, ReLU, and adaptive learning rates.

A typical generative task is as follows. At each step, a datapoint is sampled from the dataset, and part of the data is removed, and the model must infer the removed part. This is particularly clear for the denoising autoencoders and BERT.

NEURAL NETWORK ARCHITECTURES

TRAINING

During the learning phase, an unsupervised network tries to mimic the data it is given and uses the error in its mimicked output to correct itself (i.e. correct its weights and biases). Sometimes the error is expressed as a low probability that the erroneous output occurs, or it might be expressed as an unstable high energy state in the network.

In contrast to supervised methods' dominant use of backpropagation, unsupervised learning also employs other methods including: Hopfield learning rule, Boltzmann learning rule, Contrastive Divergence, Wake Sleep, Variational Inference, Maximum Likelihood, Maximum A Posteriori, Gibbs Sampling, and backpropagating reconstruction errors or hidden state reparameterizations. See the table below for more details.

ENERGY

An energy function is a macroscopic measure of a network's activation state. In Boltzmann machines, it plays the role of the Cost function. This analogy with physics is inspired by Ludwig Boltzmann's analysis of a gas' macroscopic energy from the microscopic probabilities of particle motion $p \propto e - E / k T$ {\displaystyle p\propto e^{-E/kT}} , where k is the Boltzmann constant and T is temperature. In the RBM network the relation is $p = e - E / Z$ {\displaystyle p=e^{-E}/Z} , where p {\displaystyle p} and E

{\displaystyle E} vary over every possible activation pattern and Z = ∑ All Patterns e − E (pattern) {\displaystyle \textstyle {Z=\sum _{\scriptscriptstyle {\text{All Patterns}}}e^{-E({\text{pattern}})}}} . To be more precise, p (a) = e − E (a) / Z {\displaystyle p(a)=e^{-E(a)}/Z} , where a {\displaystyle a} is an activation pattern of all neurons (visible and hidden). Hence, some early neural networks bear the name Boltzmann Machine. Paul Smolensky calls − E {\displaystyle -E\,} the Harmony. A network seeks low energy which is high Harmony.

NETWORKS

This table shows connection diagrams of various unsupervised networks, the details of which will be given in the section Comparison of Networks. Circles are neurons and edges between them are connection weights. As network design changes, features are added on to enable new capabilities or removed to make learning faster. For instance, neurons change between deterministic (Hopfield) and stochastic (Boltzmann) to allow robust output, weights are removed within a layer (RBM) to hasten learning, or connections are allowed to become asymmetric (Helmholtz).

Of the networks bearing people's names, only Hopfield worked directly with neural networks. Boltzmann and Helmholtz came before artificial neural networks, but their work in physics and physiology inspired the analytical methods that were used.

HISTORY

SPECIFIC NETWORKS

Here, we highlight some characteristics of select networks. The details of each are given in the comparison table below.

COMPARISON OF NETWORKS

HEBBIAN LEARNING, ART, SOM

The classical example of unsupervised learning in the study of neural networks is Donald Hebb's principle, that is, neurons that fire together wire together. In Hebbian learning, the connection is reinforced irrespective of an error, but is exclusively a function of the coincidence between action potentials between the two neurons. A similar version that modifies synaptic weights takes into account the time between the action potentials (spike-timing-dependent plasticity or STDP). Hebbian Learning has been hypothesized to underlie a range of cognitive functions, such as pattern recognition and experiential learning.

Among neural network models, the self-organizing map (SOM) and adaptive resonance theory (ART) are commonly used in unsupervised learning algorithms. The SOM is a topographic organization in which nearby locations in the map represent inputs with similar properties. The ART model allows the number of clusters to vary with problem size and lets the user control the degree of similarity between members of the same clusters by means of a user-defined constant called the vigilance parameter. ART networks are used for many pattern recognition tasks, such as automatic target recognition and seismic signal processing.

PROBABILISTIC METHODS

Two of the main methods used in unsupervised learning are principal component and cluster analysis. Cluster analysis is used in unsupervised learning to group, or segment, datasets with shared attributes in order to extrapolate algorithmic relationships. Cluster analysis is a branch of machine learning that groups the data that has not been labelled, classified or categorized. Instead of responding to feedback, cluster analysis identifies commonalities in the data and reacts based on the presence or

absence of such commonalities in each new piece of data. This approach helps detect anomalous data points that do not fit into either group.

A central application of unsupervised learning is in the field of density estimation in statistics, though unsupervised learning encompasses many other domains involving summarizing and explaining data features. It can be contrasted with supervised learning by saying that whereas supervised learning intends to infer a conditional probability distribution conditioned on the label of input data; unsupervised learning intends to infer an a priori probability distribution .

APPROACHES

Some of the most common algorithms used in unsupervised learning include: (1) Clustering, (2) Anomaly detection, (3) Approaches for learning latent variable models. Each approach uses several methods as follows:

METHOD OF MOMENTS

One of the statistical approaches for unsupervised learning is the method of moments. In the method of moments, the unknown parameters (of interest) in the model are related to the moments of one or more random variables, and thus, these unknown parameters can be estimated given the moments. The moments are usually estimated from samples empirically. The basic moments are first and second order moments. For a random vector, the first order moment is the mean vector, and the second order moment is the covariance matrix (when the mean is zero). Higher order moments are usually represented using tensors which are the generalization of matrices to higher orders as multi-dimensional arrays.

In particular, the method of moments is shown to be effective in learning the parameters of latent variable models. Latent variable models are statistical models where in addition to the observed variables, a set of latent variables also exists which is not observed. A highly practical example of latent variable models in machine learning is the topic modeling which is a statistical model for generating the words (observed variables) in the document based on the topic (latent variable) of the document. In the topic modeling, the words in the document are generated according to different statistical parameters when the topic of the document is changed. It is shown that method of moments (tensor decomposition techniques) consistently recover the parameters of a large class of latent variable models under some assumptions.

The Expectation–maximization algorithm (EM) is also one of the most practical methods for learning latent variable models. However, it can get stuck in local optima, and it is not guaranteed that the algorithm will converge to the true unknown parameters of the model. In contrast, for the method of moments, the global convergence is guaranteed under some conditions.

--- TABLE 1 START --- Source URL: https://en.wikipedia.org/wiki/Unsupervised_learning Columns: Hopfield, Boltzmann, RBM, Stacked Boltzmann. Row: Hopfield: A network based on magnetic domains in iron with a single self-connected layer. It can be used as a content addressable memory. | Boltzmann: Network is separated into 2 layers (hidden vs. visible), but still using symmetric 2-way weights. Following Boltzmann's thermodynamics, individual probabilities give rise to macroscopic energies. | RBM: Restricted Boltzmann Machine. This is a Boltzmann machine where lateral connections within a layer are prohibited to make analysis tractable. | Stacked Boltzmann: This network has multiple RBM's to encode a hierarchy of hidden features. After a single RBM is trained, another blue hidden layer (see left RBM) is added, and the top 2 layers are trained as a red & blue RBM. Thus the middle layers of an RBM acts as hidden or visible, depending on the training phase it is in. --- TABLE END ---

--- TABLE 2 START --- Source URL: https://en.wikipedia.org/wiki/Unsupervised_learning Columns: Helmholtz, Autoencoder, VAE. Row: Helmholtz: Instead of the bidirectional symmetric connection of the stacked Boltzmann machines, we have separate one-way connections to form a loop. It does both generation and discrimination. | Autoencoder: A feed forward network that aims to find a good middle layer representation of its input world. This network is deterministic, so it is not as robust as its successor the VAE. | VAE: Applies Variational Inference to the Autoencoder. The middle layer is a set of means & variances for Gaussian distributions. The stochastic nature allows for more robust

imagination than the deterministic autoencoder. --- TABLE END ---

--- TABLE 3 START --- Source URL: https://en.wikipedia.org/wiki/Unsupervised_learning Columns: 0, 1. Row: 0: 1974 | 1: Ising magnetic model proposed by WA Little [de] for cognition Row: 0: 1980 | 1: Kunihiko Fukushima introduces the neocognitron, which is later called a convolutional neural network. It is mostly used in SL, but deserves a mention here. Row: 0: 1982 | 1: Ising variant Hopfield net described as CAMs and classifiers by John Hopfield. Row: 0: 1983 | 1: Ising variant Boltzmann machine with probabilistic neurons described by Hinton & Sejnowski following Sherington & Kirkpatrick's 1975 work. Row: 0: 1986 | 1: Paul Smolensky publishes Harmony Theory, which is an RBM with practically the same Boltzmann energy function. Smolensky did not give a practical training scheme. Hinton did in mid-2000s. --- TABLE END ---

--- TABLE 4 START --- Source URL: https://en.wikipedia.org/wiki/Unsupervised_learning Columns: Unnamed: 0, Hopfield, Boltzmann, RBM, Stacked RBM, Helmholtz, Autoencoder, VAE. Row: Unnamed: 0: Usage & notables | Hopfield: CAM, traveling salesman problem | Boltzmann: CAM. The freedom of connections makes this network difficult to analyze. | RBM: pattern recognition. used in MNIST digits and speech. | Stacked RBM: recognition & imagination. trained with unsupervised pre-training and/or supervised fine tuning. | Helmholtz: imagination, mimicry | Autoencoder: language: creative writing, translation. vision: enhancing blurry images | VAE: generate realistic data Row: Unnamed: 0: Neuron | Hopfield: deterministic binary state. Activation = { 0 (or -1) if x is negative, 1 otherwise } | Boltzmann: stochastic binary Hopfield neuron | RBM: ← same. (extended to real-valued in mid 2000s) | Stacked RBM: ← same | Helmholtz: ← same | Autoencoder: language: LSTM. vision: local receptive fields. usually real valued relu activation. | VAE: middle layer neurons encode means & variances for Gaussians. In run mode (inference), the output of the middle layer are sampled values from the Gaussians. Row: Unnamed: 0: Connections | Hopfield: 1-layer with symmetric weights. No self-connections. | Boltzmann: 2-layers. 1-hidden & 1-visible. symmetric weights. | RBM: ← same. no lateral connections within a layer. | Stacked RBM: top layer is undirected, symmetric. other layers are 2-way, asymmetric. | Helmholtz: 3-layers: asymmetric weights. 2 networks combined into 1. | Autoencoder: 3-layers. The input is considered a layer even though it has no inbound weights. recurrent layers for NLP. feedforward convolutions for vision. input & output have the same neuron counts. | VAE: 3-layers: input, encoder, distribution sampler decoder. the sampler is not considered a layer Row: Unnamed: 0: Inference & energy | Hopfield: Energy is given by Gibbs probability measure : | Boltzmann: ← same | RBM: ← same | Stacked RBM: N/A | Helmholtz: minimize KL divergence | Autoencoder: inference is only feed-forward. previous UL networks ran forwards AND backwards | VAE: minimize error = reconstruction error - KLD Row: Unnamed: 0: Training | Hopfield: ∆wij = si*sj, for +1/-1 neuron | Boltzmann: ∆wij = e*(pij - p'ij). This is derived from minimizing KLD. e = learning rate, p' = predicted and p = actual distribution. | RBM: ∆wij = e*(< vi hj >data - < vi hj >equilibrium). This is a form of contrastive divergence w/ Gibbs Sampling. "<>" are expectations. | Stacked RBM: ← similar. train 1-layer at a time. approximate equilibrium state with a 3-segment pass. no back propagation. | Helmholtz: wake-sleep 2 phase training | Autoencoder: back propagate the reconstruction error | VAE: reparameterize hidden state for backprop --- TABLE END ---

Weak supervision (also known as semi-supervised learning) is a paradigm in machine learning, the relevance and notability of which increased with the advent of large language models due to the large amount of data required to train them. It is characterized by using a combination of a small amount of human-labeled data (exclusively used in more expensive and time-consuming supervised learning paradigm), followed by a large amount of unlabeled data (used exclusively in unsupervised learning paradigm). In other words, the desired output values are provided only for a subset of the training data. The remaining data is unlabeled or imprecisely labeled. Intuitively, it can be seen as an exam and labeled data as sample problems that the teacher solves for the class as an aid in solving another set of problems. In the transductive setting, these unsolved problems act as exam questions. In the inductive setting, they become practice problems of the sort that will make up the exam.

PROBLEM

The acquisition of labeled data for a learning problem often requires a skilled human agent (e.g. to transcribe an audio segment) or a physical experiment (e.g. determining the 3D structure of a protein or determining whether there is oil at a particular location). The cost associated with the labeling process

thus may render large, fully labeled training sets infeasible, whereas acquisition of unlabeled data is relatively inexpensive. In such situations, semi-supervised learning can be of great practical value. Semi-supervised learning is also of theoretical interest in machine learning and as a model for human learning.

TECHNIQUE

More formally, semi-supervised learning assumes a set of l {\displaystyle l} independently identically distributed examples x 1 , … , x l ∈ X {\displaystyle x_{1},\dots ,x_{l}\in X} with corresponding labels y 1 , … , y l ∈ Y {\displaystyle y_{1},\dots ,y_{l}\in Y} and u {\displaystyle u} unlabeled examples x l + 1 , … , x l + u ∈ X {\displaystyle x_{l+1},\dots ,x_{l+u}\in X} are processed. Semi-supervised learning combines this information to surpass the classification performance that can be obtained either by discarding the unlabeled data and doing supervised learning or by discarding the labels and doing unsupervised learning.

Semi-supervised learning may refer to either transductive learning or inductive learning. The goal of transductive learning is to infer the correct labels for the given unlabeled data x l + 1 , … , x l + u {\displaystyle x_{l+1},\dots ,x_{l+u}} only. The goal of inductive learning is to infer the correct mapping from X {\displaystyle X} to Y {\displaystyle Y} .

It is unnecessary (and, according to Vapnik's principle, imprudent) to perform transductive learning by way of inferring a classification rule over the entire input space; however, in practice, algorithms formally designed for transduction or induction are often used interchangeably.

ASSUMPTIONS

In order to make any use of unlabeled data, some relationship to the underlying distribution of data must exist. Semi-supervised learning algorithms make use of at least one of the following assumptions:

CONTINUITY / SMOOTHNESS ASSUMPTION

Points that are close to each other are more likely to share a label. This is also generally assumed in supervised learning and yields a preference for geometrically simple decision boundaries. In the case of semi-supervised learning, the smoothness assumption additionally yields a preference for decision boundaries in low-density regions, so few points are close to each other but in different classes.

CLUSTER ASSUMPTION

The data tend to form discrete clusters, and points in the same cluster are more likely to share a label (although data that shares a label may spread across multiple clusters). This is a special case of the smoothness assumption and gives rise to feature learning with clustering algorithms.

MANIFOLD ASSUMPTION

The data lie approximately on a manifold of much lower dimension than the input space. In this case learning the manifold using both the labeled and unlabeled data can avoid the curse of dimensionality. Then learning can proceed using distances and densities defined on the manifold.

The manifold assumption is practical when high-dimensional data are generated by some process that may be hard to model directly, but which has only a few degrees of freedom. For instance, human voice is controlled by a few vocal folds, and images of various facial expressions are controlled by a few muscles. In these cases, it is better to consider distances and smoothness in the natural space of the generating problem, rather than in the space of all possible acoustic waves or images, respectively.

HISTORY

The heuristic approach of self-training (also known as self-learning or self-labeling) is historically the oldest approach to semi-supervised learning, with examples of applications starting in the 1960s.

The transductive learning framework was formally introduced by Vladimir Vapnik in the 1970s. Interest in inductive learning using generative models also began in the 1970s. A probably approximately correct learning bound for semi-supervised learning of a Gaussian mixture was demonstrated by Ratsaby and Venkatesh in 1995.

METHODS

GENERATIVE MODELS

Generative approaches to statistical learning first seek to estimate p (x | y) {\displaystyle p(x|y)} , the distribution of data points belonging to each class. The probability p (y | x) {\displaystyle p(y|x)} that a given point x {\displaystyle x} has label y {\displaystyle y} is then proportional to p (x | y) p (y) {\displaystyle p(x|y)p(y)} by Bayes' rule. Semi-supervised learning with generative models can be viewed either as an extension of supervised learning (classification plus information about p (x) {\displaystyle p(x)}) or as an extension of unsupervised learning (clustering plus some labels).

Generative models assume that the distributions take some particular form p (x | y , θ) {\displaystyle p(x|y,\theta)} parameterized by the vector θ {\displaystyle \theta } . If these assumptions are incorrect, the unlabeled data may actually decrease the accuracy of the solution relative to what would have been obtained from labeled data alone. However, if the assumptions are correct, then the unlabeled data necessarily improves performance.

The unlabeled data are distributed according to a mixture of individual-class distributions. In order to learn the mixture distribution from the unlabeled data, it must be identifiable, that is, different parameters must yield different summed distributions. Gaussian mixture distributions are identifiable and commonly used for generative models.

The parameterized joint distribution can be written as p (x , y | θ) = p (y | θ) p (x | y , θ) {\displaystyle p(x,y|\theta)=p(y|\theta)p(x|y,\theta)} by using the chain rule. Each parameter vector θ {\displaystyle \theta } is associated with a decision function f θ (x) = argmax y p (y | x , θ) {\displaystyle f_{\theta }(x)={\underset {y}{\operatorname {argmax} }}\ p(y|x,\theta)} . The parameter is then chosen based on fit to both the labeled and unlabeled data, weighted by λ {\displaystyle \lambda } :

LOW-DENSITY SEPARATION

Another major class of methods attempts to place boundaries in regions with few data points (labeled or unlabeled). One of the most commonly used algorithms is the transductive support vector machine, or TSVM (which, despite its name, may be used for inductive learning as well). Whereas support vector machines for supervised learning seek a decision boundary with maximal margin over the labeled data, the goal of TSVM is a labeling of the unlabeled data such that the decision boundary has maximal margin over all of the data. In addition to the standard hinge loss (1 − y f (x)) + {\displaystyle (1-yf(x))_{+}} for labeled data, a loss function (1 − | f (x) |) + {\displaystyle (1-|f(x)|)_{+}} is introduced over the unlabeled data by letting y = sign ⁡ f (x) {\displaystyle y=\operatorname {sign} {f(x)}} . TSVM then selects f ∗ (x) = h ∗ (x) + b {\displaystyle f^{*}(x)=h^{*}(x)+b} from a reproducing kernel Hilbert space H {\displaystyle {\mathcal {H}}} by minimizing the regularized empirical risk:

An exact solution is intractable due to the non-convex term (1 − | f (x) |) + {\displaystyle (1-|f(x)|)_{+}} , so research focuses on useful approximations.

Other approaches that implement low-density separation include Gaussian process models, information regularization, and entropy minimization (of which TSVM is a special case).

LAPLACIAN REGULARIZATION

Laplacian regularization has been historically approached through graph-Laplacian. Graph-based methods for semi-supervised learning use a graph representation of the data, with a node for each labeled and unlabeled example. The graph may be constructed using domain knowledge or similarity of examples; two common methods are to connect each data point to its k {\displaystyle k} nearest neighbors or to examples within some distance ■ {\displaystyle \epsilon } . The weight W i j {\displaystyle W_{ij}} of an edge between x i {\displaystyle x_{i}} and x j {\displaystyle x_{j}} is then set to e – ■ x i – x j ■ 2 / ■ 2 {\displaystyle e^{-\|x_{i}-x_{j}\|^{2}/\epsilon ^{2}}} .

Within the framework of manifold regularization, the graph serves as a proxy for the manifold. A term is added to the standard Tikhonov regularization problem to enforce smoothness of the solution relative to the manifold (in the intrinsic space of the problem) as well as relative to the ambient input space. The minimization problem becomes

where H {\displaystyle {\mathcal {H}}} is a reproducing kernel Hilbert space and M {\displaystyle {\mathcal {M}}} is the manifold on which the data lie. The regularization parameters λ A {\displaystyle \lambda _{A}} and λ I {\displaystyle \lambda _{I}} control smoothness in the ambient and intrinsic spaces respectively. The graph is used to approximate the intrinsic regularization term. Defining the graph Laplacian L = D − W {\displaystyle L=D-W} where D i i = ∑ j = 1 l + u W i j {\displaystyle D_{ii}=\sum _{j=1}^{l+u}W_{ij}} and f {\displaystyle \mathbf {f} } is the vector [f (x 1) … f (x l + u)] {\displaystyle } , we have

The graph-based approach to Laplacian regularization is to put in relation with finite difference method.

The Laplacian can also be used to extend the supervised learning algorithms: regularized least squares and support vector machines (SVM) to semi-supervised versions Laplacian regularized least squares and Laplacian SVM.

HEURISTIC APPROACHES

Some methods for semi-supervised learning are not intrinsically geared to learning from both unlabeled and labeled data, but instead make use of unlabeled data within a supervised learning framework. For instance, the labeled and unlabeled examples x 1 , … , x l + u {\displaystyle x_{1},\dots ,x_{l+u}} may inform a choice of representation, distance metric, or kernel for the data in an unsupervised first step. Then supervised learning proceeds from only the labeled examples. In this vein, some methods learn a low-dimensional representation using the supervised data and then apply either low-density separation or graph-based methods to the learned representation. Iteratively refining the representation and then performing semi-supervised learning on said representation may further improve performance.

Self-training is a wrapper method for semi-supervised learning. First a supervised learning algorithm is trained based on the labeled data only. This classifier is then applied to the unlabeled data to generate more labeled examples as input for the supervised learning algorithm. Generally only the labels the classifier is most confident in are added at each step. In natural language processing, a common self-training algorithm is the Yarowsky algorithm for problems like word sense disambiguation, accent restoration, and spelling correction.

Co-training is an extension of self-training in which multiple classifiers are trained on different (ideally disjoint) sets of features and generate labeled examples for one another.

IN HUMAN COGNITION

Human responses to formal semi-supervised learning problems have yielded varying conclusions about the degree of influence of the unlabeled data. More natural learning problems may also be viewed as instances of semi-supervised learning. Much of human concept learning involves a small amount of direct instruction (e.g. parental labeling of objects during childhood) combined with large amounts of unlabeled experience (e.g. observation of objects without naming or counting them, or at least without feedback).

Human infants are sensitive to the structure of unlabeled natural categories such as images of dogs and cats or male and female faces. Infants and children take into account not only unlabeled examples, but the sampling process from which labeled examples arise.

WEAK SUPERVISION IN PREDICTIVE MAINTENANCE

Weak supervision is an emerging machine learning approach in predictive maintenance that addresses the challenge of limited or imprecise labeled data. Traditional predictive maintenance models often rely on large volumes of accurately labeled failure and operational data, which can be costly or impractical to obtain. Weak supervision mitigates this by leveraging imperfect sources of supervision—such as noisy labels, heuristics, domain expert rules, or partially labeled datasets—to train predictive models. By incorporating techniques such as data programming, label modeling, and semi-supervised learning, weak supervision enables the development of robust predictive maintenance systems capable of identifying equipment failures or anomalies with reduced reliance on high-quality labeled data. This approach is particularly valuable in industrial settings where machine failures are rare and labeled fault data is scarce .

Self-supervised learning (SSL) is a paradigm in machine learning where a model is trained on a task using the data itself to generate supervisory signals, rather than relying on externally-provided labels. In the context of neural networks, self-supervised learning aims to leverage inherent structures or relationships within the input data to create meaningful training signals. SSL tasks are designed so that solving them requires capturing essential features or relationships in the data. The input data is typically augmented or transformed in a way that creates pairs of related samples, where one sample serves as the input, and the other is used to formulate the supervisory signal. This augmentation can involve introducing noise, cropping, rotation, or other transformations. Self-supervised learning more closely imitates the way humans learn to classify objects.

During SSL, the model learns in two steps. First, the task is solved based on an auxiliary or pretext classification task using pseudo-labels, which help to initialize the model parameters. Next, the actual task is performed with supervised or unsupervised learning.

Self-supervised learning has produced promising results in recent years, and has found practical application in fields such as audio processing, and is being used by Facebook and others for speech recognition.

PSEUDO-LABELS

Pseudo-labels are automatically generated labels that a model assigns to unlabeled data based on its own predictions. They are widely used in self-supervised and semi-supervised learning, where ground-truth annotations are limited or unavailable. By treating predicted labels as surrogate ground truth, learning algorithms can make use of large quantities of unlabeled data in the training process.

Pseudo-labeling also plays an important role in systems that must adapt to concept drift, where the statistical properties of the data change over time. In these scenarios, the model may detect that an incoming instance deviates from previously learned behavior. The system then generates a classification result for that instance, and this predicted class is used as a pseudo-label for updating or retraining model components that are becoming outdated. This approach enables continuous adaptation in dynamic environments without requiring manual annotation.

In many adaptive learning pipelines, pseudo-labels are chosen when the classifier produces sufficiently confident predictions, reducing the risk of propagating errors. These pseudo-labeled instances are then incorporated into training to refresh or evolve the model's understanding of emerging data patterns, particularly when existing components show signs of “aging” due to drift or distributional shifts. This strategy reduces reliance on manual labeling while helping maintain long-term model performance.

TYPES

AUTOASSOCIATIVE SELF-SUPERVISED LEARNING

Autoassociative self-supervised learning is a specific category of self-supervised learning where a neural network is trained to reproduce or reconstruct its own input data. In other words, the model is tasked with learning a representation of the data that captures its essential features or structure, allowing it to regenerate the original input.

The term "autoassociative" comes from the fact that the model is essentially associating the input data with itself. This is often achieved using autoencoders, which are a type of neural network architecture used for representation learning. Autoencoders consist of an encoder network that maps the input data to a lower-dimensional representation (latent space), and a decoder network that reconstructs the input from this representation.

The training process involves presenting the model with input data and requiring it to reconstruct the same data as closely as possible. The loss function used during training typically penalizes the difference between the original input and the reconstructed output (e.g. mean squared error). By minimizing this reconstruction error, the autoencoder learns a meaningful representation of the data in its latent space.

CONTRASTIVE SELF-SUPERVISED LEARNING

For a binary classification task, training data can be divided into positive examples and negative examples. Positive examples are those that match the target. For example, if training a classifier to identify birds, the positive training data would include images that contain birds. Negative examples would be images that do not. Contrastive self-supervised learning uses both positive and negative examples. The loss function in contrastive learning is used to minimize the distance between positive sample pairs, while maximizing the distance between negative sample pairs.

An early example uses a pair of 1-dimensional convolutional neural networks to process a pair of images and maximize their agreement.

Contrastive Language-Image Pre-training (CLIP) allows joint pretraining of a text encoder and an image encoder, such that a matching image-text pair have image encoding vector and text encoding vector that span a small angle (having a large cosine similarity).

InfoNCE (Noise-Contrastive Estimation) is a method to optimize two models jointly, based on Noise Contrastive Estimation (NCE). Given a set X = { x 1 , … x N } {\displaystyle X=\left\{x_{1},\ldots x_{N}\right\}} of N {\displaystyle N} random samples containing one positive sample from p (x t + k ■ c t) {\displaystyle p\left(x_{t+k}\mid c_{t}\right)} and N − 1 {\displaystyle N-1} negative samples from the 'proposal' distribution p (x t + k) {\displaystyle p\left(x_{t+k}\right)} , it minimizes the following loss function:

L N = − E X [log ■ f k (x t + k , c t) ∑ x j ∈ X f k (x j , c t)] {\displaystyle {\mathcal {L}}_{\mathrm {N} }=-\mathbb {E} _{X}\left }

NON-CONTRASTIVE SELF-SUPERVISED LEARNING

Non-contrastive self-supervised learning (NCSSL) uses only positive examples. Counterintuitively, NCSSL converges on a useful local minimum rather than reaching a trivial solution, with zero loss. For the example of binary classification, it would trivially learn to classify each example as positive. Effective NCSSL requires an extra predictor on the online side that does not back-propagate on the target side.

COMPARISON WITH OTHER FORMS OF MACHINE LEARNING

SSL belongs to supervised learning methods insofar as the goal is to generate a classified output from the input. At the same time, however, it does not require the explicit use of labeled input-output pairs. Instead, correlations, metadata embedded in the data, or domain knowledge present in the input are implicitly and autonomously extracted from the data. These supervisory signals, extracted from the data, can then be used for training.

SSL is similar to unsupervised learning in that it does not require labels in the sample data. Unlike unsupervised learning, however, learning is not done using inherent data structures.

Semi-supervised learning combines supervised and unsupervised learning, requiring only a small portion of the learning data be labeled.

In transfer learning, a model designed for one task is reused on a different task.

Training an autoencoder intrinsically constitutes a self-supervised process, because the output pattern needs to become an optimal reconstruction of the input pattern itself. However, in current jargon, the term 'self-supervised' often refers to tasks based on a pretext-task training setup. This involves the (human) design of such pretext task(s), unlike the case of fully self-contained autoencoder training.

In reinforcement learning, self-supervising learning from a combination of losses can create abstract representations where only the most important information about the state are kept in a compressed way.

EXAMPLES

Self-supervised learning is particularly suitable for speech recognition. For example, Facebook developed wav2vec, a self-supervised algorithm, to perform speech recognition using two deep convolutional neural networks that build on each other.

Google's Bidirectional Encoder Representations from Transformers (BERT) model is used to better understand the context of search queries.

OpenAI's GPT-3 is an autoregressive language model that can be used in language processing. It can be used to translate texts or answer questions, among other things.

Bootstrap Your Own Latent (BYOL) is a NCSSL that produced excellent results on ImageNet and on transfer and semi-supervised benchmarks.

The Yarowsky algorithm is an example of self-supervised learning in natural language processing. From a small number of labeled examples, it learns to predict which word sense of a polysemous word is being used at a given point in text.

DirectPred is a NCSSL that directly sets the predictor weights instead of learning it via typical gradient descent.

Self-GenomeNet is an example of self-supervised learning in genomics.

Self-supervised learning continues to gain prominence as a new approach across diverse fields. Its ability to leverage unlabeled data effectively opens new possibilities for advancement in machine learning, especially in data-driven application domains.

In machine learning and optimal control, reinforcement learning (RL) is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. Reinforcement learning is one of the three basic machine learning paradigms, alongside supervised learning and unsupervised learning.

While supervised learning and unsupervised learning algorithms respectively attempt to discover patterns in labeled and unlabeled data, reinforcement learning involves training an agent through interactions with its environment. To learn to maximize rewards from these interactions, the agent makes decisions between trying new actions to learn more about the environment (exploration), or using current knowledge of the environment to take the best action (exploitation). The search for the optimal balance between these two strategies is known as the exploration–exploitation dilemma.

The environment is typically stated in the form of a Markov decision process, as many reinforcement learning algorithms use dynamic programming techniques. The main difference between classical

dynamic programming methods and reinforcement learning algorithms is that the latter do not assume knowledge of an exact mathematical model of the Markov decision process, and they target large Markov decision processes where exact methods become infeasible.

PRINCIPLES

Due to its generality, reinforcement learning is studied in many disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimization, multi-agent systems, swarm intelligence, and statistics. In the operations research and control literature, RL is called approximate dynamic programming, or neuro-dynamic programming. The problems of interest in RL have also been studied in the theory of optimal control, which is concerned mostly with the existence and characterization of optimal solutions, and algorithms for their exact computation, and less with learning or approximation (particularly in the absence of a mathematical model of the environment).

Basic reinforcement learning is modeled as a Markov decision process:

The purpose of reinforcement learning is for the agent to learn an optimal (or near-optimal) policy that maximizes the reward function or other user-provided reinforcement signal that accumulates from immediate rewards. This is similar to processes that appear to occur in animal psychology. For example, biological brains are hardwired to interpret signals such as pain and hunger as negative reinforcements, and interpret pleasure and food intake as positive reinforcements. In some circumstances, animals learn to adopt behaviors that optimize these rewards. This suggests that animals are capable of reinforcement learning.

A basic reinforcement learning agent interacts with its environment in discrete time steps. At each time step t, the agent receives the current state S t {\displaystyle S_{t}} and reward R t {\displaystyle R_{t}} . It then chooses an action A t {\displaystyle A_{t}} from the set of available actions, which is subsequently sent to the environment. The environment moves to a new state S t + 1 {\displaystyle S_{t+1}} and the reward R t + 1 {\displaystyle R_{t+1}} associated with the transition (S t , A t , S t + 1) {\displaystyle (S_{t},A_{t},S_{t+1})} is determined. The goal of a reinforcement learning agent is to learn a policy:

π : S × A → [0 , 1] π (s , a) = Pr (A t = a ■ S t = s) {\displaystyle {\begin{aligned}&\pi :{\mathcal {S}}\times {\mathcal {A}}\to \\&\pi (s,a)=\Pr(A_{t}{=}a\mid S_{t}{=}s)\end{aligned}}}

that maximizes the expected cumulative reward.

Formulating the problem as a Markov decision process assumes the agent directly observes the current environmental state; in this case, the problem is said to have full observability. If the agent only has access to a subset of states, or if the observed states are corrupted by noise, the agent is said to have partial observability, and formally the problem must be formulated as a partially observable Markov decision process. In both cases, the set of actions available to the agent can be restricted. For example, the state of an account balance could be restricted to be positive; if the current value of the state is 3 and the state transition attempts to reduce the value by 4, the transition will not be allowed.

When the agent's performance is compared to that of an agent that acts optimally, the difference in performance yields the notion of regret. In order to act near optimally, the agent must reason about long-term consequences of its actions (i.e., maximize future rewards), although the immediate reward associated with this might be negative.

Thus, reinforcement learning is particularly well-suited to problems that include a long-term versus short-term reward trade-off. It has been applied successfully to various problems, including energy storage, robot control, photovoltaic generators, backgammon, checkers, Go (AlphaGo), and autonomous driving systems.

Two elements make reinforcement learning powerful: the use of samples to optimize performance, and the use of function approximation to deal with large environments. Thanks to these two key components, RL can be used in large environments in the following situations:

The first two of these problems could be considered planning problems (since some form of model is available), while the last one could be considered to be a genuine learning problem. However,

reinforcement learning converts both planning problems to machine learning problems.

EXPLORATION

The trade-off between exploration and exploitation has been most thoroughly studied through the multi-armed bandit problem and for finite state space Markov decision processes in Burnetas and Katehakis (1997).

Reinforcement learning requires clever exploration mechanisms; randomly selecting actions, without reference to an estimated probability distribution, shows poor performance. The case of (small) finite Markov decision processes is relatively well understood. However, due to the lack of algorithms that scale well with the number of states (or scale to problems with infinite state spaces), simple exploration methods are the most practical.

One such method is ε {\displaystyle \varepsilon } -greedy, where 0 < ε < 1 {\displaystyle 0<\varepsilon <1} is a parameter controlling the amount of exploration vs. exploitation. With probability 1 − ε {\displaystyle 1-\varepsilon } , exploitation is chosen, and the agent chooses the action that it believes has the best long-term effect (ties between actions are broken uniformly at random). Alternatively, with probability ε {\displaystyle \varepsilon } , exploration is chosen, and the action is chosen uniformly at random. ε {\displaystyle \varepsilon } is usually a fixed parameter but can be adjusted either according to a schedule (making the agent explore progressively less), or adaptively based on heuristics.

ALGORITHMS FOR CONTROL LEARNING

Even if the issue of exploration is disregarded and even if the state was observable (assumed hereafter), the problem remains to use past experience to find out which actions lead to higher cumulative rewards.

CRITERION OF OPTIMALITY

The agent's action selection is modeled as a map called policy: π : A × S → [0 , 1] π (a , s) = Pr (A t = a ■ S t = s) {\displaystyle {\begin{aligned}&\pi :{\mathcal {A}}\times {\mathcal {S}}\to \\&\pi (a,s)=\Pr(A_{t}{=}a\mid S_{t}{=}s)\end{aligned}}}

The policy map gives the probability of taking action a {\displaystyle a} when in state s {\displaystyle s} . There are also deterministic policies π {\displaystyle \pi } for which π (s) {\displaystyle \pi (s)} denotes the action that should be played at state s {\displaystyle s} .

The state-value function V π (s) {\displaystyle V_{\pi }(s)} is defined as, expected discounted return starting with state s {\displaystyle s} , i.e. S 0 = s {\displaystyle S_{0}=s} , and successively following policy π {\displaystyle \pi } . Hence, roughly speaking, the value function estimates "how good" it is to be in a given state.

V π (s) = E ■ [G ■ S 0 = s] = E ■ [∑ t = 0 ∞ γ t R t + 1 ■ S 0 = s] , {\displaystyle V_{\pi }(s)=\operatorname {\mathbb {E} } =\operatorname {\mathbb {E} } \left ,}

where the random variable G {\displaystyle G} denotes the discounted return, and is defined as the sum of future discounted rewards:

G = ∑ t = 0 ∞ γ t R t + 1 = R 1 + γ R 2 + γ 2 R 3 + ■ , {\displaystyle G=\sum _{t=0}^{\infty }\gamma ^{t}R_{t+1}=R_{1}+\gamma R_{2}+\gamma ^{2}R_{3}+\cdots ,}

where R t + 1 {\displaystyle R_{t+1}} is the reward for transitioning from state S t {\displaystyle S_{t}} to S t + 1 {\displaystyle S_{t+1}} , 0 ≤ γ < 1 {\displaystyle 0\leq \gamma <1} is the discount rate. γ {\displaystyle \gamma } is less than 1, so rewards in the distant future are weighted less than rewards in the immediate future.

The algorithm must find a policy with maximum expected discounted return. From the theory of Markov decision processes it is known that, without loss of generality, the search can be restricted to the set of so-called stationary policies. A policy is stationary if the action-distribution returned by it depends only

on the last state visited (from the observation agent's history). The search can be further restricted to deterministic stationary policies. A deterministic stationary policy deterministically selects actions based on the current state. Since any such policy can be identified with a mapping from the set of states to the set of actions, these policies can be identified with such mappings with no loss of generality.

BRUTE FORCE

The brute force approach entails two steps:

One problem with this is that the number of policies can be large, or even infinite. Another is that the variance of the returns may be large, which requires many samples to accurately estimate the discounted return of each policy.

These problems can be ameliorated if we assume some structure and allow samples generated from one policy to influence the estimates made for others. The two main approaches for achieving this are value function estimation and direct policy search.

VALUE FUNCTION

Value function approaches attempt to find a policy that maximizes the discounted return by maintaining a set of estimates of expected discounted returns E ■ [G] {\displaystyle \operatorname {\mathbb {E} } } for some policy (usually either the "current" or the optimal one).

These methods rely on the theory of Markov decision processes, where optimality is defined in a sense stronger than the one above: A policy is optimal if it achieves the best-expected discounted return from any initial state (i.e., initial distributions play no role in this definition). Again, an optimal policy can always be found among stationary policies.

To define optimality in a formal manner, define the state-value of a policy π {\displaystyle \pi } by

V π (s) = E ■ [G ■ s , π] , {\displaystyle V^{\pi }(s)=\operatorname {\mathbb {E} } ,}

where G {\displaystyle G} stands for the discounted return associated with following π {\displaystyle \pi } from the initial state s {\displaystyle s} . Defining V ∗ (s) {\displaystyle V^{*}(s)} as the maximum possible state-value of V π (s) {\displaystyle V^{\pi }(s)} , where π {\displaystyle \pi } is allowed to change,

V ∗ (s) = max π V π (s) . {\displaystyle V^{*}(s)=\max _{\pi }V^{\pi }(s).}

A policy that achieves these optimal state-values in each state is called optimal. Clearly, a policy that is optimal in this sense is also optimal in the sense that it maximizes the expected discounted return, since V ∗ (s) = max π E [G ■ s , π] {\displaystyle V^{*}(s)=\max _{\pi }\mathbb {E} } , where s {\displaystyle s} is a state randomly sampled from the distribution μ {\displaystyle \mu } of initial states (so μ (s) = Pr (S 0 = s) {\displaystyle \mu (s)=\Pr(S_{0}=s)}).

Although state-values suffice to define optimality, it is useful to define action-values. Given a state s {\displaystyle s} , an action a {\displaystyle a} and a policy π {\displaystyle \pi } , the action-value of the pair (s , a) {\displaystyle (s,a)} under π {\displaystyle \pi } is defined by

Q π (s , a) = E ■ [G ■ s , a , π] , {\displaystyle Q^{\pi }(s,a)=\operatorname {\mathbb {E} } ,}

where G {\displaystyle G} now stands for the random discounted return associated with first taking action a {\displaystyle a} in state s {\displaystyle s} and following π {\displaystyle \pi } , thereafter.

The theory of Markov decision processes states that if π ∗ {\displaystyle \pi ^{*}} is an optimal policy, we act optimally (take the optimal action) by choosing the action from Q π ∗ (s , ⋅) {\displaystyle Q^{\pi ^{*}}(s,\cdot)} with the highest action-value at each state, s {\displaystyle s} . The action-value function of such an optimal policy (Q π ∗ {\displaystyle Q^{\pi ^{*}}}) is called the optimal action-value function and is commonly denoted by Q ∗ {\displaystyle Q^{*}} . In summary, the knowledge of the optimal action-value function alone suffices to know how to act optimally.

Assuming full knowledge of the Markov decision process, the two basic approaches to compute the optimal action-value function are value iteration and policy iteration. Both algorithms compute a sequence of functions Q k {\displaystyle Q_{k}} (k = 0 , 1 , 2 , … {\displaystyle k=0,1,2,\ldots }) that converge to Q ∗ {\displaystyle Q^{*}} . Computing these functions involves computing expectations over

the whole state-space, which is impractical for all but the smallest (finite) Markov decision processes. In reinforcement learning methods, expectations are approximated by averaging over samples and using function approximation techniques to cope with the need to represent value functions over large state-action spaces.

Monte Carlo methods are used to solve reinforcement learning problems by averaging sample returns. Unlike methods that require full knowledge of the environment's dynamics, Monte Carlo methods rely solely on actual or simulated experience—sequences of states, actions, and rewards obtained from interaction with an environment. This makes them applicable in situations where the complete dynamics are unknown. Learning from actual experience does not require prior knowledge of the environment and can still lead to optimal behavior. When using simulated experience, only a model capable of generating sample transitions is required, rather than a full specification of transition probabilities, which is necessary for dynamic programming methods.

Monte Carlo methods apply to episodic tasks, where experience is divided into episodes that eventually terminate. Policy and value function updates occur only after the completion of an episode, making these methods incremental on an episode-by-episode basis, though not on a step-by-step (online) basis. The term "Monte Carlo" generally refers to any method involving random sampling; however, in this context, it specifically refers to methods that compute averages from complete returns, rather than partial returns.

These methods function similarly to the bandit algorithms, in which returns are averaged for each state-action pair. The key difference is that actions taken in one state affect the returns of subsequent states within the same episode, making the problem non-stationary. To address this non-stationarity, Monte Carlo methods use the framework of general policy iteration (GPI). While dynamic programming computes value functions using full knowledge of the Markov decision process, Monte Carlo methods learn these functions through sample returns. The value functions and policies interact similarly to dynamic programming to achieve optimality, first addressing the prediction problem and then extending to policy improvement and control, all based on sampled experience.

The first problem is corrected by allowing the procedure to change the policy (at some or all states) before the values settle. This too may be problematic as it might prevent convergence. Most current algorithms do this, giving rise to the class of generalized policy iteration algorithms. Many actor-critic methods belong to this category.

The second issue can be corrected by allowing trajectories to contribute to any state-action pair in them. This may also help to some extent with the third problem, although a better solution when returns have high variance is Sutton's temporal difference (TD) methods that are based on the recursive Bellman equation. The computation in TD methods can be incremental (when after each transition the memory is changed and the transition is thrown away), or batch (when the transitions are batched and the estimates are computed once based on the batch). Batch methods, such as the least-squares temporal difference method, may use the information in the samples better, while incremental methods are the only choice when batch methods are infeasible due to their high computational or memory complexity. Some methods try to combine the two approaches. Methods based on temporal differences also overcome the fourth issue.

Another problem specific to TD comes from their reliance on the recursive Bellman equation. Most TD methods have a so-called λ {\displaystyle \lambda } parameter (0 ≤ λ ≤ 1) {\displaystyle (0\leq \lambda \leq 1)} that can continuously interpolate between Monte Carlo methods that do not rely on the Bellman equations and the basic TD methods that rely entirely on the Bellman equations. This can be effective in palliating this issue.

In order to address the fifth issue, function approximation methods are used. Linear function approximation starts with a mapping φ {\displaystyle \phi } that assigns a finite-dimensional vector to each state-action pair. Then, the action values of a state-action pair (s , a) {\displaystyle (s,a)} are obtained by linearly combining the components of φ (s , a) {\displaystyle \phi (s,a)} with some weights θ {\displaystyle \theta } :

Q (s , a) = ∑ i = 1 d θ i φ i (s , a) . {\displaystyle Q(s,a)=\sum _{i=1}^{d}\theta _{i}\phi _{i}(s,a).}

The algorithms then adjust the weights, instead of adjusting the values associated with the individual state-action pairs. Methods based on ideas from nonparametric statistics (which can be seen to construct their own features) have been explored.

Value iteration can also be used as a starting point, giving rise to the Q-learning algorithm and its many variants. Including Deep Q-learning methods when a neural network is used to represent Q, with various applications in stochastic search problems.

The problem with using action-values is that they may need highly precise estimates of the competing action values that can be hard to obtain when the returns are noisy, though this problem is mitigated to some extent by temporal difference methods. Using the so-called compatible function approximation method compromises generality and efficiency.

DIRECT POLICY SEARCH

An alternative method is to search directly in (some subset of) the policy space, in which case the problem becomes a case of stochastic optimization. The two approaches available are gradient-based and gradient-free methods.

Gradient-based methods (policy gradient methods) start with a mapping from a finite-dimensional (parameter) space to the space of policies: given the parameter vector θ {\displaystyle \theta } , let $\pi \theta$ {\displaystyle \pi _{\theta }} denote the policy associated to θ {\displaystyle \theta } . Defining the performance function by $\rho (\theta) = \rho \pi \theta$ {\displaystyle \rho (\theta)=\rho ^{\pi _{\theta }}} under mild conditions this function will be differentiable as a function of the parameter vector θ {\displaystyle \theta } . If the gradient of ρ {\displaystyle \rho } was known, one could use gradient ascent. Since an analytic expression for the gradient is not available, only a noisy estimate is available. Such an estimate can be constructed in many ways, giving rise to algorithms such as Williams's REINFORCE method (which is known as the likelihood ratio method in the simulation-based optimization literature).

A large class of methods avoids relying on gradient information. These include simulated annealing, cross-entropy search or methods of evolutionary computation. Many gradient-free methods can achieve (in theory and in the limit) a global optimum.

Policy search methods may converge slowly given noisy data. For example, this happens in episodic problems when the trajectories are long and the variance of the returns is large. Value-function based methods that rely on temporal differences might help in this case. In recent years, actor–critic methods have been proposed and performed well on various problems.

Policy search methods have been used in the robotics context. Many policy search methods may get stuck in local optima (as they are based on local search).

MODEL-BASED ALGORITHMS

Finally, all of the above methods can be combined with algorithms that first learn a model of the Markov decision process, the probability of each next state given an action taken from an existing state. For instance, the Dyna algorithm learns a model from experience, and uses that to provide more modelled transitions for a value function, in addition to the real transitions. Such methods can sometimes be extended to use of non-parametric models, such as when the transitions are simply stored and "replayed" to the learning algorithm.

Model-based methods can be more computationally intensive than model-free approaches, and their utility can be limited by the extent to which the Markov decision process can be learnt.

There are other ways to use models than to update a value function. For instance, in model predictive control the model is used to update the behavior directly.

THEORY

Both the asymptotic and finite-sample behaviors of most algorithms are well understood. Algorithms with provably good online performance (addressing the exploration issue) are known.

Efficient exploration of Markov decision processes is given in Burnetas and Katehakis (1997). Finite-time performance bounds have also appeared for many algorithms, but these bounds are expected to be rather loose and thus more work is needed to better understand the relative advantages

and limitations.

For incremental algorithms, asymptotic convergence issues have been settled.
Temporal-difference-based algorithms converge under a wider set of conditions than was previously possible (for example, when used with arbitrary, smooth function approximation).

RESEARCH

Research topics include:

COMPARISON OF KEY ALGORITHMS

The following table lists the key algorithms for learning a policy depending on several criteria:

ASSOCIATIVE REINFORCEMENT LEARNING

Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.

DEEP REINFORCEMENT LEARNING

This approach extends reinforcement learning by using a deep neural network and without explicitly designing the state space. The work on learning ATARI games by Google DeepMind increased attention to deep reinforcement learning or end-to-end reinforcement learning.

ADVERSARIAL DEEP REINFORCEMENT LEARNING

Adversarial deep reinforcement learning is an active area of research in reinforcement learning focusing on vulnerabilities of learned policies. In this research area some studies initially showed that reinforcement learning policies are susceptible to imperceptible adversarial manipulations. While some methods have been proposed to overcome these susceptibilities, in the most recent studies it has been shown that these proposed solutions are far from providing an accurate representation of current vulnerabilities of deep reinforcement learning policies.

FUZZY REINFORCEMENT LEARNING

By introducing fuzzy inference in reinforcement learning, approximating the state-action value function with fuzzy rules in continuous space becomes possible. The IF - THEN form of fuzzy rules make this approach suitable for expressing the results in a form close to natural language. Extending FRL with Fuzzy Rule Interpolation allows the use of reduced size sparse fuzzy rule-bases to emphasize cardinal rules (most important state-action values).

INVERSE REINFORCEMENT LEARNING

In inverse reinforcement learning (IRL), no reward function is given. Instead, the reward function is inferred given an observed behavior from an expert. The idea is to mimic observed behavior, which is often optimal or close to optimal. One popular IRL paradigm is named maximum entropy inverse reinforcement learning (MaxEnt IRL). MaxEnt IRL estimates the parameters of a linear model of the reward function by maximizing the entropy of the probability distribution of observed trajectories subject to constraints related to matching expected feature counts. Recently it has been shown that MaxEnt IRL is a particular case of a more general framework named random utility inverse reinforcement learning (RU-IRL). RU-IRL is based on random utility theory and Markov decision processes. While prior IRL approaches assume that the apparent random behavior of an observed agent is due to it

following a random policy, RU-IRL assumes that the observed agent follows a deterministic policy but randomness in observed behavior is due to the fact that an observer only has partial access to the features the observed agent uses in decision making. The utility function is modeled as a random variable to account for the ignorance of the observer regarding the features the observed agent actually considers in its utility function.

MULTI-OBJECTIVE REINFORCEMENT LEARNING

Multi-objective reinforcement learning (MORL) is a form of reinforcement learning concerned with conflicting alternatives. It is distinct from multi-objective optimization in that it is concerned with agents acting in environments.

SAFE REINFORCEMENT LEARNING

Safe reinforcement learning (SRL) can be defined as the process of learning policies that maximize the expectation of the return in problems in which it is important to ensure reasonable system performance and/or respect safety constraints during the learning and/or deployment processes. An alternative approach is risk-averse reinforcement learning, where instead of the expected return, a risk-measure of the return is optimized, such as the conditional value at risk (CVaR). In addition to mitigating risk, the CVaR objective increases robustness to model uncertainties. However, CVaR optimization in risk-averse RL requires special care, to prevent gradient bias and blindness to success.

SELF-REINFORCEMENT LEARNING

Self-reinforcement learning (or self-learning), is a learning paradigm which does not use the concept of immediate reward R a (s , s ′) {\displaystyle R_{a}(s,s')} after transition from s {\displaystyle s} to s ′ {\displaystyle s'} with action a {\displaystyle a} . It does not use an external reinforcement, it only uses the agent internal self-reinforcement. The internal self-reinforcement is provided by mechanism of feelings and emotions. In the learning process emotions are backpropagated by a mechanism of secondary reinforcement. The learning equation does not include the immediate reward, it only includes the state evaluation.

The self-reinforcement algorithm updates a memory matrix W = ■ w (a , s) ■ {\displaystyle W=\|w(a,s)\|} such that in each iteration executes the following machine learning routine:

Initial conditions of the memory are received as input from the genetic environment. It is a system with only one input (situation), and only one output (action, or behavior).

Self-reinforcement (self-learning) was introduced in 1982 along with a neural network capable of self-reinforcement learning, named Crossbar Adaptive Array (CAA). The CAA computes, in a crossbar fashion, both decisions about actions and emotions (feelings) about consequence states. The system is driven by the interaction between cognition and emotion.

REINFORCEMENT LEARNING IN NATURAL LANGUAGE PROCESSING

In recent years, reinforcement learning has become a significant concept in natural language processing (NLP), where tasks are often sequential decision-making rather than static classification. Reinforcement learning is where an agent take actions in an environment to maximize the accumulation of rewards. This framework is best fit for many NLP tasks, including dialogue generation, text summarization, and machine translation, where the quality of the output depends on optimizing long-term or human-centered goals rather than the prediction of single correct label.

Early application of RL in NLP emerged in dialogue systems, where conversation was determined as a series of actions optimized for fluency and coherence. These early attempts, including policy gradient and sequence-level training techniques, laid a foundation for the broader application of reinforcement learning to other areas of NLP.

A major breakthrough happened with the introduction of reinforcement learning from human feedback (RLHF), a method in which human feedback ratings are used to train a reward model that guides the

RL agent. Unlike traditional rule-based or supervised systems, RLHF allows models to align their behavior with human judgments on complex and subjective tasks. This technique was initially used in the development of InstructGPT, an effective language model trained to follow human instructions and later in ChatGPT which incorporates RLHF for improving output responses and ensuring safety.

More recently, researchers have explored the use of offline RL in NLP to improve dialogue systems without the need of live human interaction. These methods optimize for user engagement, coherence, and diversity based on past conversation logs and pre-trained reward models.

One of examples is DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. This model was trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step.

STATISTICAL COMPARISON OF REINFORCEMENT LEARNING ALGORITHMS

Efficient comparison of RL algorithms is essential for research, deployment and monitoring of RL systems. To compare different algorithms on a given environment, an agent can be trained for each algorithm. Since the performance is sensitive to implementation details, all algorithms should be implemented as closely as possible to each other. After the training is finished, the agents can be run on a sample of test episodes, and their scores (returns) can be compared. Since episodes are typically assumed to be i.i.d, standard statistical tools can be used for hypothesis testing, such as T-test and permutation test. This requires to accumulate all the rewards within an episode into a single number—the episodic return. However, this causes a loss of information, as different time-steps are averaged together, possibly with different levels of noise. Whenever the noise level varies across the episode, the statistical power can be improved significantly, by weighting the rewards according to their estimated noise.

CHALLENGES AND LIMITATIONS

Despite significant advancements, reinforcement learning (RL) continues to face several challenges and limitations that hinder its widespread application in real-world scenarios.

SAMPLE INEFFICIENCY

RL algorithms often require a large number of interactions with the environment to learn effective policies, leading to high computational costs and time-intensive to train the agent. For instance, OpenAI's Dota-playing bot utilized thousands of years of simulated gameplay to achieve human-level performance. Techniques like experience replay and curriculum learning have been proposed to deprive sample inefficiency, but these techniques add more complexity and are not always sufficient for real-world applications.

STABILITY AND CONVERGENCE ISSUES

Training RL models, particularly for deep neural network-based models, can be unstable and prone to divergence. A small change in the policy or environment can lead to extreme fluctuations in performance, making it difficult to achieve consistent results. This instability is further enhanced in the case of the continuous or high-dimensional action space, where the learning step becomes more complex and less predictable.

GENERALIZATION AND TRANSFERABILITY

The RL agents trained in specific environments often struggle to generalize their learned policies to new, unseen scenarios. This is the major setback preventing the application of RL to dynamic real-world environments where adaptability is crucial. The challenge is to develop such algorithms that can transfer knowledge across tasks and environments without extensive retraining.

BIAS AND REWARD FUNCTION ISSUES

Designing appropriate reward functions is critical in RL because poorly designed reward functions can lead to unintended behaviors. In addition, RL systems trained on biased data may perpetuate existing biases and lead to discriminatory or unfair outcomes. Both of these issues requires careful consideration of reward structures and data sources to ensure fairness and desired behaviors.

--- TABLE 1 START --- Source URL: https://en.wikipedia.org/wiki/Reinforcement_learning Columns: Algorithm, Description, Policy, Action space, State space, Operator. Row: Algorithm: Monte Carlo | Description: Every visit to Monte Carlo | Policy: Either | Action space: Discrete | State space: Discrete | Operator: Sample-means of state-values or action-values Row: Algorithm: TD learning | Description: State–action–reward–state | Policy: Off-policy | Action space: Discrete | State space: Discrete | Operator: State-value Row: Algorithm: Q-learning | Description: State–action–reward–state | Policy: Off-policy | Action space: Discrete | State space: Discrete | Operator: Action-value Row: Algorithm: SARSA | Description: State–action–reward–state–action | Policy: On-policy | Action space: Discrete | State space: Discrete | Operator: Action-value Row: Algorithm: DQN | Description: Deep Q Network | Policy: Off-policy | Action space: Discrete | State space: Continuous | Operator: Action-value --- TABLE END ---